Table of Contents



CHROMA TECHNICAL REPORT

July 14, 2025

Context Rot: How Increasing Input Tokens Impacts LLM Performance

Kelly Hong Researcher - Chroma

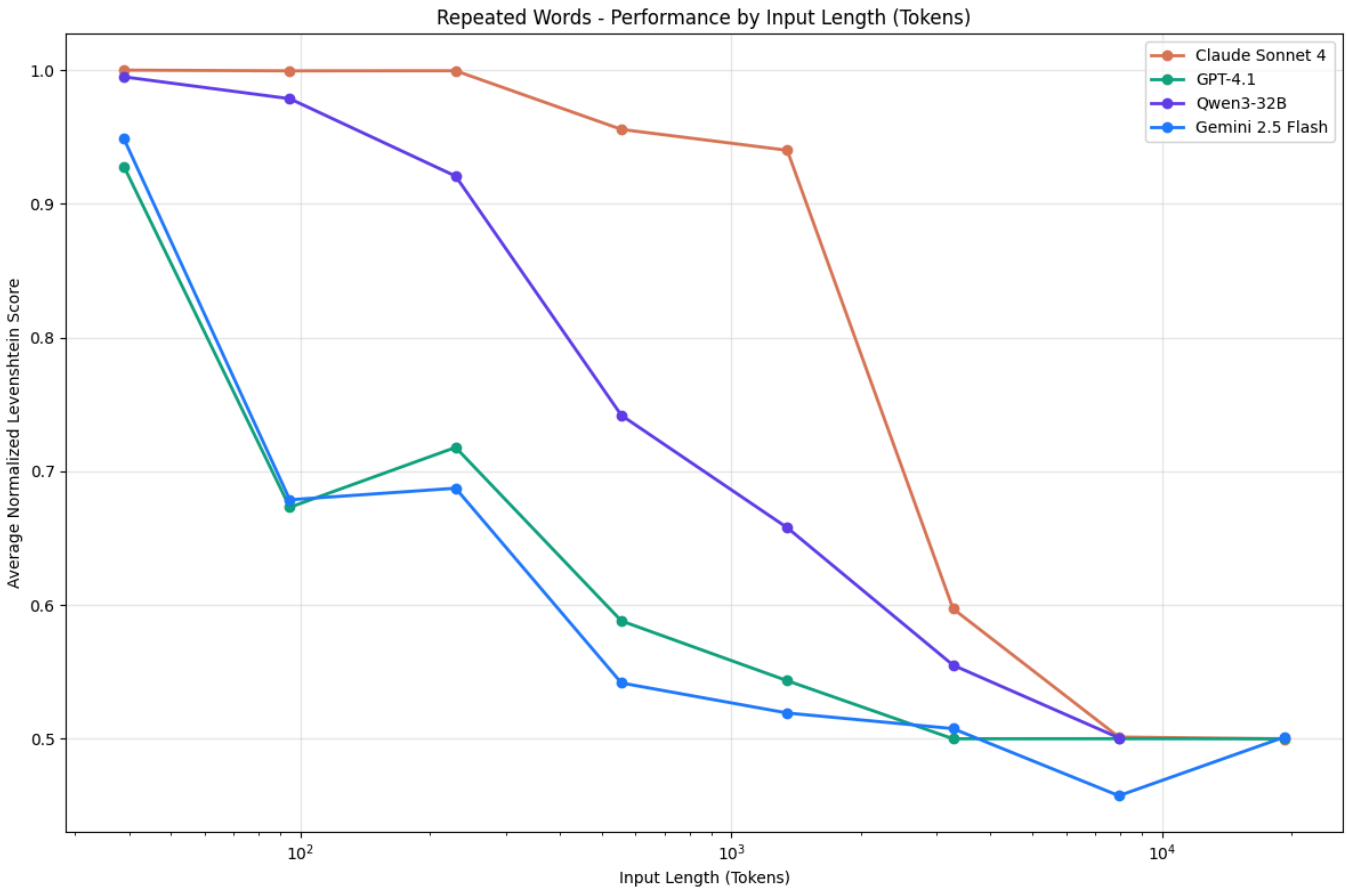
Anton Troynikov Cofounder, Advisor - Chroma

Jeff Huber Cofounder, CEO - Chroma

Large Language Models (LLMs) are typically presumed to process context uniformly—that is, the model should handle the 10,000th token just as reliably as the 100th. However,



In this report, we evaluate 18 LLMs, including the state-of-the-art GPT-4.1, Claude 4, Gemini 2.5, and Qwen3 models. Our results reveal that models do not use their context uniformly; instead, their performance grows increasingly unreliable as input length grows.

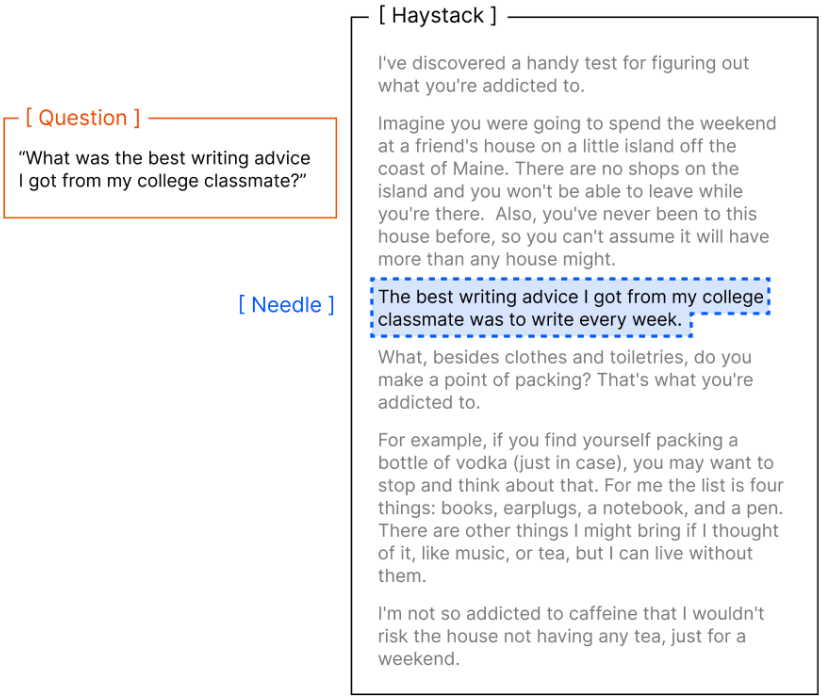


Claude Sonnet 4, GPT-4.1, Qwen3-32B, and Gemini 2.5 Flash on Repeated Words Task

Recent developments in LLMs show a trend toward longer context windows, with the input token count of the latest models reaching the millions. Because these models achieve near-perfect scores on widely adopted benchmarks like Needle in a Haystack (NIAH) [1], it's often assumed that their performance is uniform across long-context tasks.



direct lexical matching, which may not be representative of flexible, semantically oriented tasks.



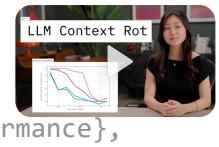
Example Needle in a Haystack (NIAH) Setup

We extend the standard NIAH task, to investigate model behavior in previously underexplored settings. We examine the effects of needles with semantic, rather than direct lexical matches, as well as the effects of introducing variations to the haystack content.

Additionally, we include a conversational question-answer evaluation using LongMemEval [2], as well as a synthetic task in which models replicate a series of repeated words. Each task remains intentionally simple and is deliberately controlled to isolate the impact of context length alone.

We demonstrate that even under these minimal conditions, model performance degrades as input length increases, often in surprising and non-uniform ways. Real-world applications typically involve much greater complexity, implying that the influence of input length may be even more pronounced in practice.

Our in-depth technical report continues below. If you find our work useful, please consider citing us:




```
    title = {Context Rot: How Increasing Input Tokens Impacts LLM Performance},
    author = {Hong, Kelly and Troynikov, Anton and Huber, Jeff},
    year = {2025},
    month = {July},
    institution = {Chroma},
    url = {https://research.trychroma.com/context-rot},
}
```

Interested in working on improving retrieval for AI applications? Chroma is Hiring

---

Introduction

It is common for modern LLMs to have input context lengths in the millions of tokens. Gemini 1.5 Pro [3] first introduced their 1M context window in early 2024, followed by the recent GPT-4.1's 1M context window [4] and Llama 4 with 10M [5]. The use case for long context is compelling: longer context means that the LLM can process more information with each call and generate more informed outputs.

Long context evaluations for these models often demonstrate consistent performance across input lengths. However, these evaluations are narrow in scope and not representative of how long context is used in practice. The most commonly used test, Needle in a Haystack (NIAH), is a simple lexical retrieval task often used to generalize a model's ability to reliably handle long context. Real applications, such as agent tasks or summarization, demand significantly more processing and reasoning over broader, often more ambiguous information.

Designing realistic long context benchmarks is challenging. Tasks often grow in complexity as input length increases, making it difficult to isolate whether performance drops are due to longer inputs or inherently harder problems. To address this, our experiments hold task complexity constant while varying only the input length—allowing us to directly measure the effect of input length alone.



We present the following:

- An evaluation across 18 LLMs, including leading closed-source and open-weights models, revealing nonuniform performance with increasing input length.
- A writeup of observed model-specific behavior patterns when handling distractors and varying question-answer similarity.
- The **complete codebase** to replicate our results.

Related Work

One of the most widely used benchmarks for evaluating a model's long context capabilities is Needle in a Haystack (NIAH). While useful as a scalable test, it measures a narrow capability: lexical retrieval. Models typically perform well on NIAH, which has led to the perception that long-context is largely solved.

However, NIAH underestimates what most long context tasks require in practice. Variants of NIAH, like NoLiMa [6] which include needle-question pairs with non-lexical matches, reveal significant performance drops. Other tasks that appear similar in regards to difficulty, such as AbsenceBench [7] which tests models for recognizing the absence of a given snippet of text, also demonstrate performance degradation with growing input length.

Additionally, long context tasks often involve disambiguating amongst distractors as part of the task. One example is Multi-round co-reference resolution (MRCR) [8] [9], which involves retrieving the i-th instance of a specific user ask, amongst similar user asks, in a multi-turn conversation. However, there remains a lack of investigation into the impact of distractors in long context settings.

An important factor in long-context tasks is how input length is scaled. Latent List [8] is a task in which the model must perform a fixed number of Python list operations across various input lengths. Various ways to fill irrelevant context are tested, which reveal non-uniform impact on model performance [1]. For instance, adding list



length.

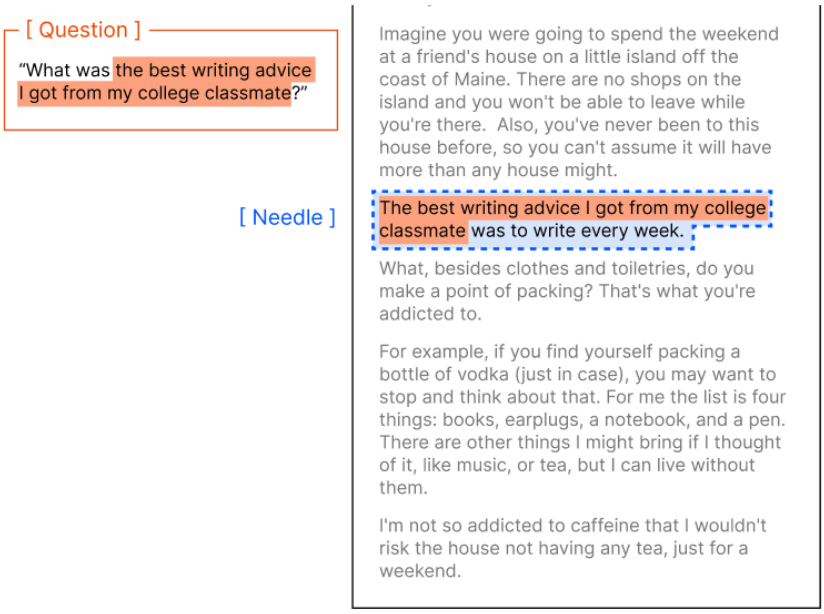
Similarly, Graphwalks [10] is a graph traversal task in which the model is given a directed graph composed of hexadecimal hashes, then asked to perform breadth-first search starting from a random node. Increasing input length means increasing the size of the graph to traverse through, which increases task difficulty as a result. It is difficult to disambiguate increasing task complexity from input length, which makes it difficult to isolate the impact on performance due to input length alone. This points to the importance of isolating input length as the variable of interest, which is essential for understanding of how LLMs actually behave with long inputs.

Needle in a Haystack Extension

The classic Needle in a Haystack task involves placing a random fact (the 'needle') in the middle of a long context window (the 'haystack'), then asking the model about that fact.

The original implementation of this task uses a needle-question pair with lexical matches. However, usage of long context in practice often requires semantic understanding of ambiguous tasks.





Example Needle in a Haystack (NIAH) Setup with Lexical Matching

NoLiMa has demonstrated non-lexical matching to be a challenge for models as context length increases. This task utilizes needle-question pairs that require models to infer latent associations, for example:

```
Question: Which character has been to Helsinki?

Needle: Actually, Yuki lives next to the Kiasma museum.
```

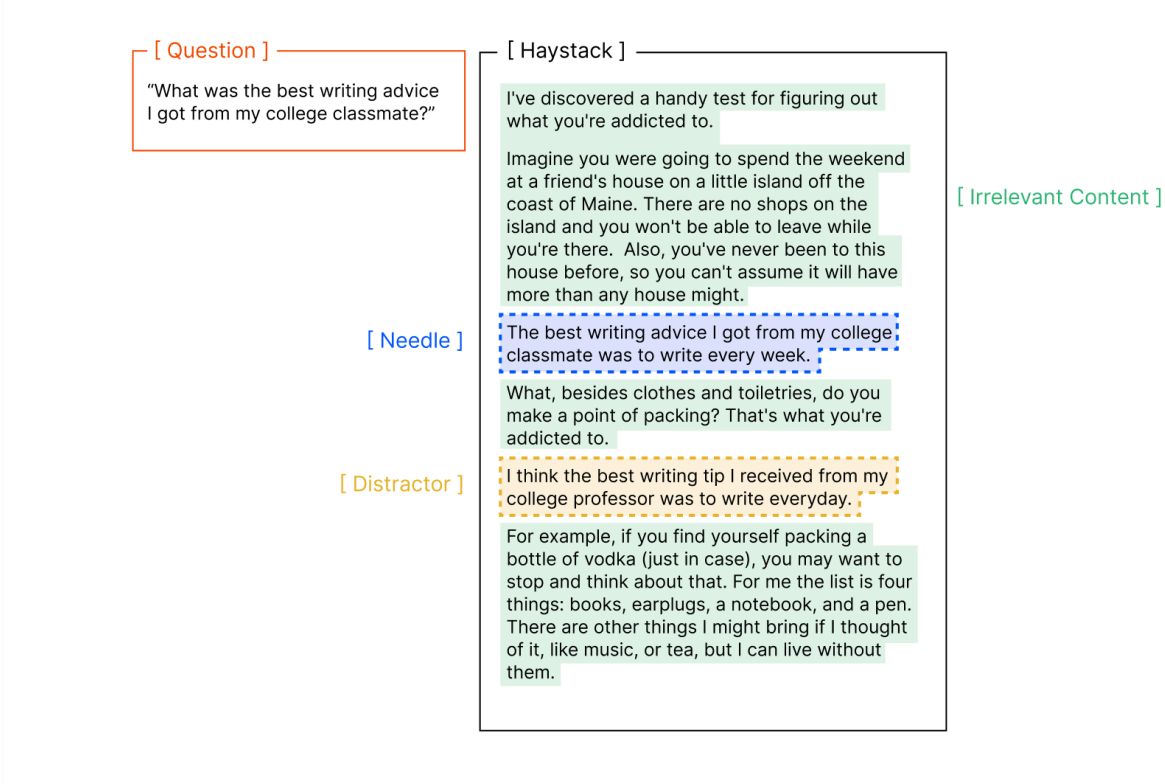
NoLiMa - sample Needle-Question pair

In order to answer this question, the model would first have to know that Kiasma museum is located in Helsinki, then make that latent association link. This tests the model not only for its non-lexical matching abilities, but also for its world knowledge. 72.4% of needle-question pairs from NoLiMa require such external knowledge, making this benchmark closer to a test of how models handle both tasks at once rather than pure non-lexical matching alone.

Testing the impact of non-lexical matching in isolation remains underexplored. Furthermore, this binary distinction of "lexical" versus "non-lexical" oversimplifies the complexity of question-answering in real-world scenarios. Needle-question pairs exist on a spectrum of similarity, yet they are all classified under these broad categories.



Throughout this report, we distinguish between distractors and irrelevant content:



Comparison - Distractor vs. Irrelevant Context

- Distractors are topically related to the needle, but do not quite answer the question
- Irrelevant content is unrelated to the needle and question

Prior work has demonstrated that distractors have non-uniform impact, yet most evaluations involve short input lengths and older models. Current state-of-the-art models are claimed to be more resilient to distractors, yet their performance has not been extensively tested across various input lengths.

Another underexplored aspect of NIAH is the haystack itself, which is often simply treated as a means of scaling input length, but this assumes that the haystack content itself has no effect on task performance. If the model is indeed insensitive to the content of the haystack, then varying this content, for example the haystack's topic or narrative flow, should have no influence on the results. However, this assumption remains largely untested.

We design four controlled experiments to investigate the influence of these factors:



We compute the cosine similarity between needle-question pairs using embeddings. For robustness, we average across five embedding models: text-embedding-3-small, text-embedding-3-large, jina-embeddings-v3, voyage-3-large, and all-MiniLM-L6-v2. We measure how model performance is impacted by needle-question similarity as input length increases.

Impact of Distractors

Taking a high-similarity needle-question pair, we write four distractors. We have the following setups:

- Baseline: needle only, no distractors
- Single distractor: needle + one randomly positioned distractor
- Multiple distractors: needle + all four distractors randomly positioned

We test the impact of distractors on model performance as input length increases to measure non-uniformity amongst distractors and input lengths.

Needle-Haystack Similarity

We use two thematically distinct haystacks, Paul Graham essays and arXiv papers [12], and write corresponding needles for each. To measure needle-haystack similarity, we embed the haystack and retrieve the top-5 chunks for each needle, then average their cosine similarity scores. This process is repeated across five different embedding models for robustness.

Haystack Structure

In typical NIAH setups, haystacks are concatenations of coherent texts, each with their own logical flow of ideas. For instance, the original NIAH benchmark uses a series of Paul Graham essays, where each essay follows a structured organization of ideas to form an argument. To evaluate whether this structure influences model performance, we compare two conditions:

- Original: preserves the natural flow of ideas within each excerpt
- Shuffled: sentences are randomly reordered throughout the haystack to maintain the same overall topic without logical continuity



increasing input length.

- Lower similarity needle-question pairs increases the rate of performance degradation.
- Distractors have non-uniform impact on model performance with regards to how distracting they are relative to each other. We see this impact more prominently as input length increases, and observe distinctions in how various models respond to them.
- Needle-haystack similarity does not have a uniform effect on model performance, suggesting the need for further investigation.
- The structural pattern of the haystack consistently shows an impact on how models process long inputs.

Details

For every unique combination of needle type, haystack topic, and haystack structure, we test each model across:

- 8 input lengths
- 11 needle positions

We evaluate each model across its maximum context window with temperature=0 unless that setting is incompatible (i.e. o3) or explicitly discouraged (i.e. Qwen's "thinking mode"). For Qwen models, we apply the YaRN method [13] to extend from 32,768 to 131,072 tokens.

We include models in both standard and "thinking mode" where applicable.

We evaluate model outputs using an aligned GPT-4.1 judge, using our method outlined in the appendix.

We note some rare instances of a model refusing to attempt the task (69 out of 194,480 total LLM calls—0.035%). For example, Claude Opus 4 may sometimes have an empty output with stop_reason="refusal".

Needle-Question Similarity



specify precise keywords for relevant parts. Instead, the model must infer relevance.

We vary the similarity of our needle-question pairs, quantified by the cosine similarity of their embeddings. We find that as needle-question similarity decreases, model performance degrades more significantly with increasing input length. This reflects more realistic scenarios where exact question-answer matches are rare, and semantic ambiguity compounds the challenge of long input processing.

Experiment

We source our haystack content from two domains: Paul Graham essays (as in the original NIAH experiment), and arXiv papers. For each haystack topic (PG essays, arXiv), we first determine common themes to guide our question and needle writing.

We use clustering to identify the most common topics that appear for a given corpus:

1. Chunk documents into 1-3 sentence chunks
2. Embed each chunk using text-embedding-3-large
3. Use UMAP [14] for dimensionality reduction with the following parameters: n_neighbors=30, min_dist=0.05, n_components=50, random_state=42
4. Use HDBSCAN [15] to create clusters with the following parameters: min_cluster_size=10, min_samples=15
5. Get 20 representative chunks for the largest clusters using maximal marginal relevance (MMR)
6. Manually examine the largest clusters to determine their themes and style

Using this method, we identify writing advice as a common topic for PG essays, often in anecdotal form. For arXiv papers, we identify information retrieval as a common topic, specifically re-ranking.

We write a corresponding question for each topic:

```
PG essays: "What was the best writing advice I got from my college classmate?"

arXiv papers: "Which low-latency reranker is preferred for scientific domains?"
```



Before writing our needles, we verify that answers to these questions do not exist in the haystack content:

1. We store our previously computed haystack chunk embeddings in a vector database.
2. Query top-10 results from that vector database with our question embedding.
3. Manually examine these results to verify that they do not answer the given question.

This sets up a fair testing environment as it ensures that alternative answers do not exist, and any incorrect answers are due to model hallucinations.

For each question, we write 8 needles that each belong to the large cluster which we verify using approximate predictions. Needles that belong to the writing/retrieval cluster with >0.9 probability are considered to topically blend into the haystack. We manually write these needles to avoid data contamination.

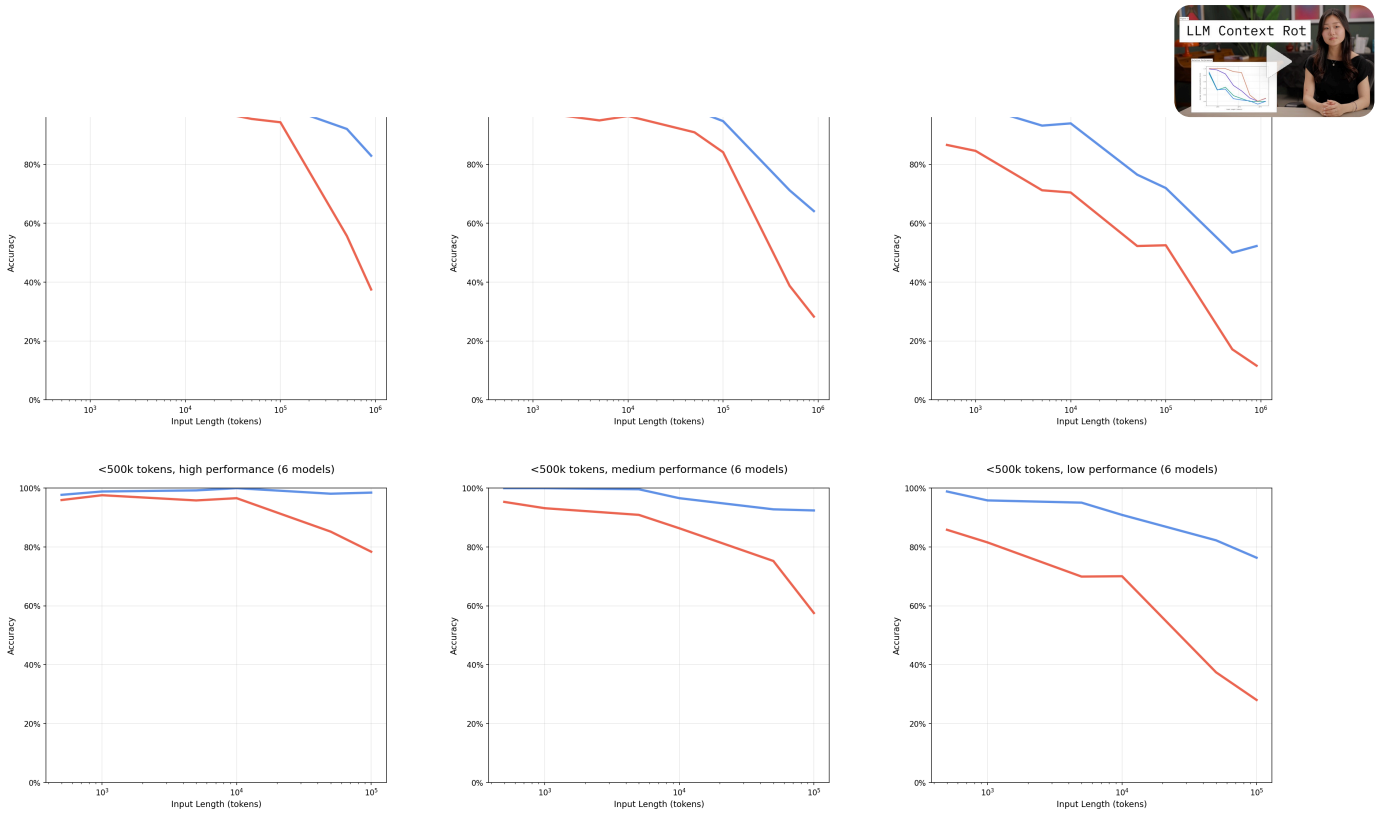
For the 8 needles, we also vary the level of ambiguity, quantified through the following method:

1. Using an embedding model, we compute embeddings for needle and question and their cosine similarity.
2. Repeat across five embedding models (text-embedding-3-small, text-embedding-3-large, jina-embeddings-v3, voyage-3-large, and all-MiniLM-L6-v2).

For the PG essays topic, our needles range from 0.445-0.775 needle-question similarity with <0.1 standard deviation across the five embedding models. For the arXiv topic, we have a needle-question similarity range of 0.521-0.829, also with <0.1 standard deviation.

Results

We observe a clear pattern that performance degrades more quickly in input length with lower similarity needle-question pairs.



NIAH: Needle-Question Similarity (thinking/non-thinking modes of the same model are treated separately) - arXiv haystack/arXiv needles
High Performance: upper 33% performance
Blue: high-similarity needles (upper 50% similarity)
Red: low-similarity needles (lower 50% similarity)

At short input lengths, the models perform well even on low-similarity pairs. We see this most clearly in the high/medium-performance models, demonstrating that these models are capable of succeeding at this task for all needle-question pairs.

The observed performance degradation at longer input lengths is not due to the intrinsic difficulty of the needle-question pairing. By holding the needle-question pair fixed and varying only the amount of irrelevant content, we isolate input size as the primary factor in performance decline.

We also examine whether needle position influences performance. Testing across 11 needle positions, we find no notable variation in performance for this specific NIAH task.

Impact of Distractors

It has already been established with older models that distractors degrade model performance and have non-uniform impact. Newer models are claimed to reliably handle any distractor, but does this hold true as input length increases?



with ambiguity.

Experiment

From each haystack topic (PG essays and arXiv papers), we take a needle with high needle-question similarity (second highest out of eight), and manually write 4 distractors:

```
Question: "What was the best writing advice I got from my college classmate?"

Needle: "I think the best writing tip I received from my college classmate was to
write every week."

Distractors:
```

- "The best writing tip I received from my college professor was to write everyday."
- "The worst writing advice I got from my college classmate was to write each essay in five different styles."
- "The best writing advice I got from my classmate was to write each essay in three different styles, this was back in high school."
- "I thought the best writing advice I got from my college classmate was to write each essay in four different styles, but not anymore."

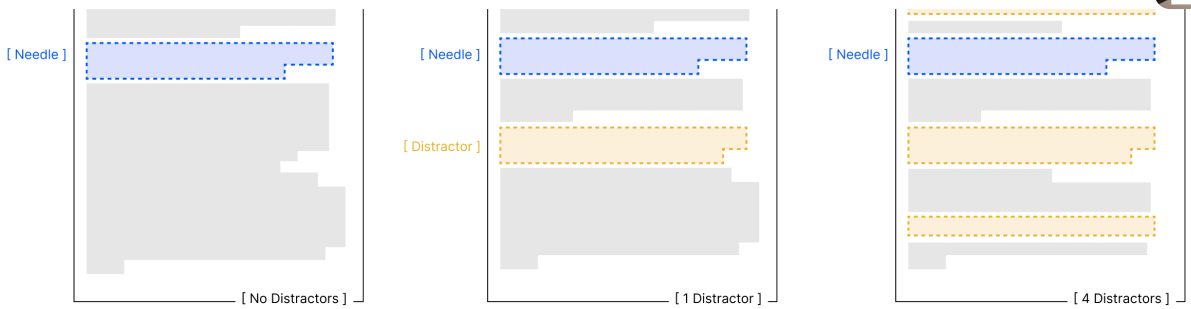
Distractors for Paul Graham Essay Topic & Needle with High Needle-Question Similarity

Instead of testing all eight needles with distractors, we use one needle with high needle-question similarity to create a condition in which the needle should be relatively easy to identify. We see from previous results that models generally perform well on this needle across input lengths due to high needle-question similarity, which allows us to better isolate and measure the impact of distractors alone.

We run three test conditions:

- No distractors (baseline): Needle only
- Single distractor: Needle + one distractor (randomly positioned)
- Multiple distractors: Needle + all four distractors, randomly positioned throughout the haystack

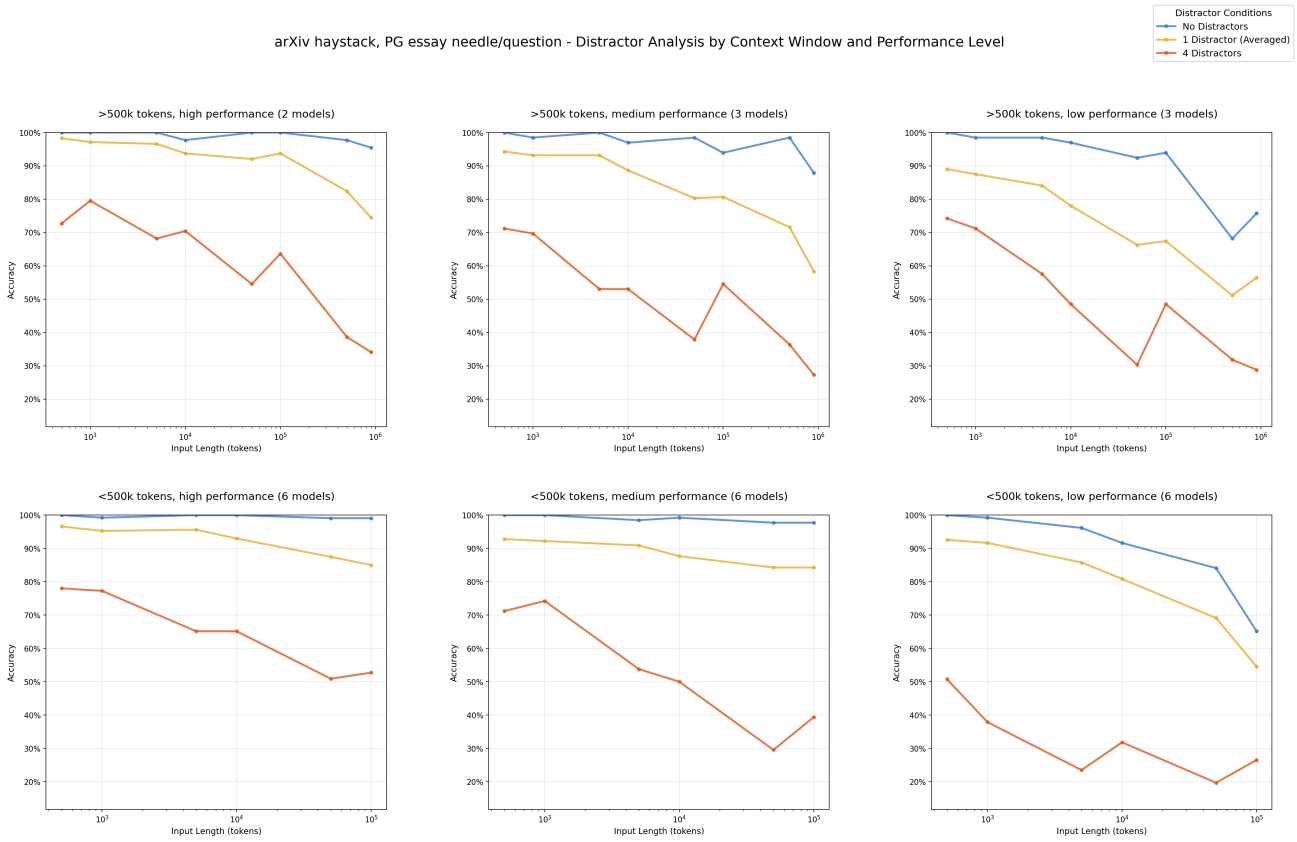




Impact of Distractors - Three Conditions

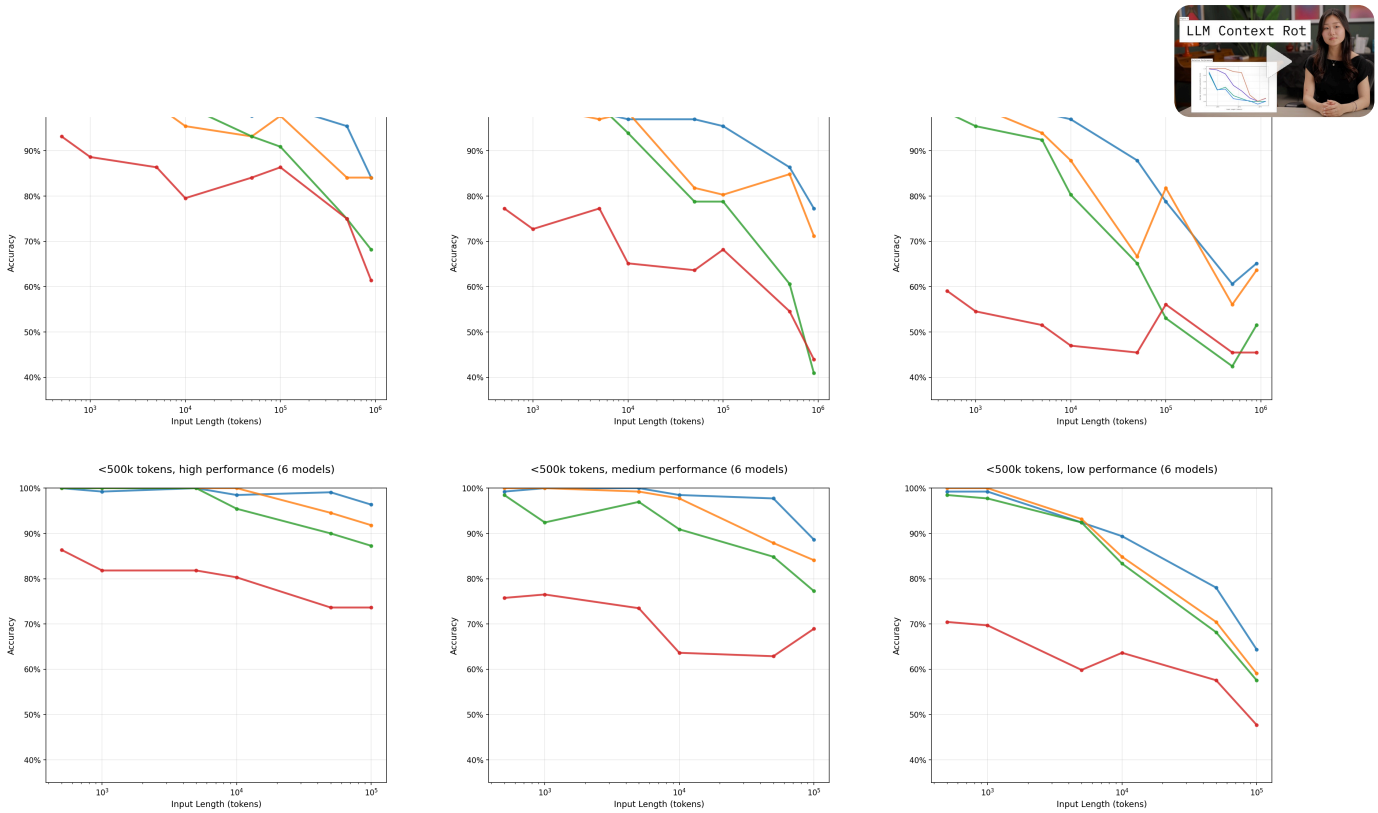
Results

Even a single distractor reduces performance relative to the baseline (needle only), and adding four distractors compounds this degradation further.



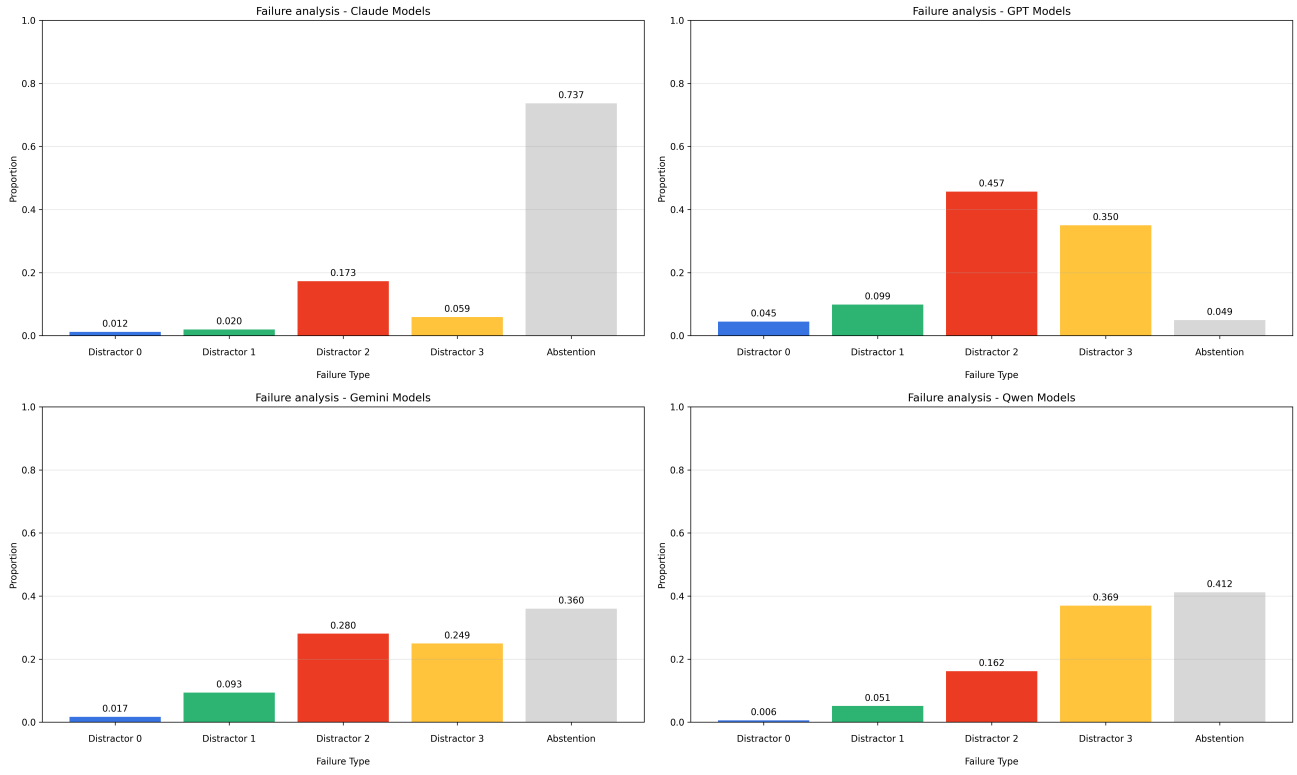
Impact of Distractors: Performance by Number of Distractors - arXiv haystack/PG essay needles

We are also able to see that distractors do not have uniform impact. For example, in our arXiv haystack and PG essay needle combination, we can see that distractor 3 (red) causes greater performance decline relative to the other distractors.



Impact of Distractors: Performance by Individual Distractors - arXiv haystack/PG essay needles

To further investigate this non-uniform impact, we analyze the failed attempts of various models in the 4-distractor condition. For the arXiv haystack and PG essay needle combination, we see that distractors 2 and 3 appear most frequently in hallucinated responses across models.



Impact of Distractors: Failure Analysis - arXiv haystack/PG essay needles



explicitly stating that no answer can be found. In contrast, GPT models show the highest rates of hallucination, often generating confident but incorrect responses when distractors are present.

Needle-Haystack Similarity

In long-context tasks, irrelevant context is often treated as a neutral placeholder to scale up input length. It's typically assumed that the content of this irrelevant context doesn't matter, as long as it doesn't directly interfere with the task.

However, a natural question arises: does the needle-haystack similarity influence task difficulty at all? Intuitively, if the needle blends in with the content of the haystack, the model may have greater difficulty in extracting the needle.

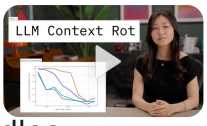
Our findings reveal that needle-haystack similarity has a non-uniform effect on model performance.

Experiment

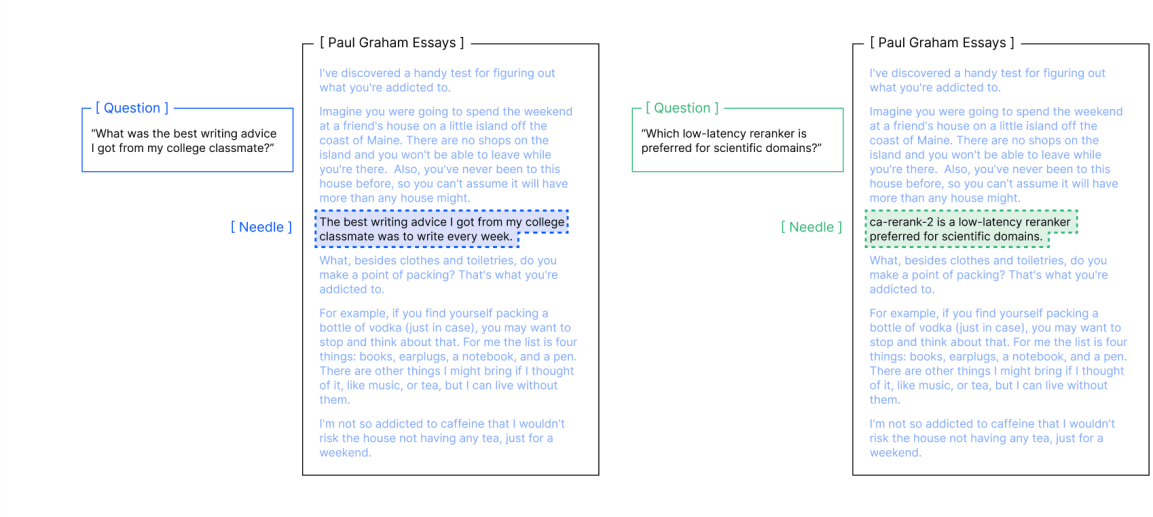
Using the needles from our needle-question similarity experiment, we set up our experiment to test the impact of needle-haystack similarity.

We measure needle-haystack similarity by embedding the haystack and retrieving the top five most similar chunks for each needle, then averaging their cosine similarity scores. This process is repeated across five different embedding models for robustness.

In the PG essay haystack, PG essay needles have an average needle-haystack similarity score of 0.529 with a variation of 0.101, while arXiv needles average 0.368 needle-haystack similarity with a variation of 0.111. Conversely, in the arXiv haystack, arXiv needles average 0.654 needle-haystack similarity with a variation of 0.0858,



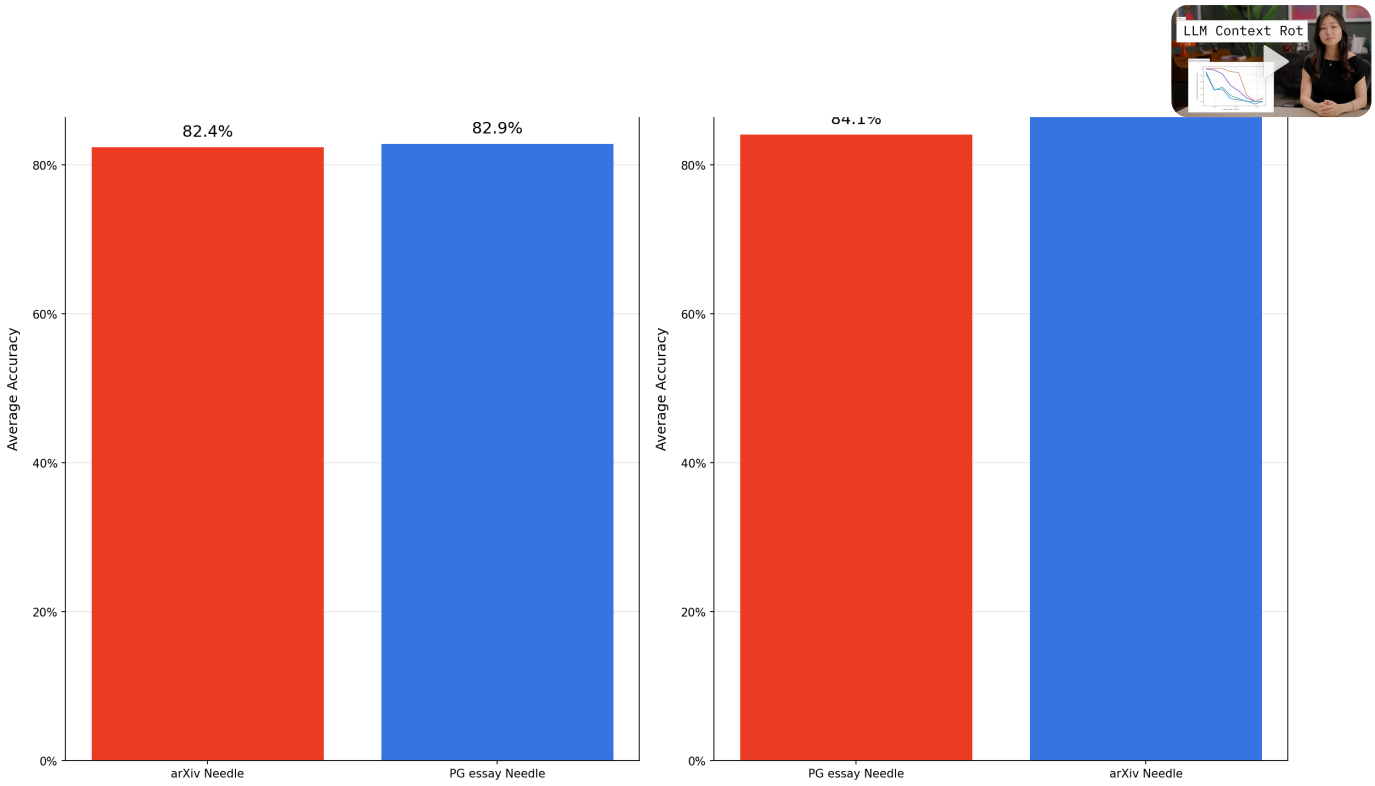
On each haystack, we test semantically similar needles against unrelated needles. For instance, we place both PG essay and arXiv needles within a Paul Graham essay haystack to compare the two conditions:



Needle-Haystack Similarity: Experimental Setup

Results

We test both PG essay and arXiv needles in two haystack types: Paul Graham essays and arXiv papers. In the Paul Graham essay haystack, arXiv needles perform significantly better relative to the PG essay needles; in other words, models perform better when the needle does not semantically blend in with its haystack. In the arXiv haystack, however, we observe only minimal performance differences between our arXiv and PG essay needles.



Needle-Haystack Similarity Results

Testing across only two topics is insufficient to draw a generalizable conclusion that higher needle-haystack similarity degrades model performance on this task. This does highlight, however, the non-uniform nature of long-context processing. Even when task structure and needle-question similarity are held constant, changing the semantic similarity between the needle and the haystack can influence results. This points to an underexplored area in long-context benchmarks and a meaningful direction for future research.

Haystack Structure

Aside from needle-haystack similarity, we also consider the structural pattern of the haystack.

If the haystack is composed of coherent essays, a randomly inserted needle may disrupt the logical flow of ideas, making it more noticeable. In contrast, in a shuffled haystack of randomly ordered sentences, the needle may blend in more easily since the overall context lacks structure. This follows the assumption that models are sensitive to the logical flow of context—processing it in a structured, order-sensitive manner.

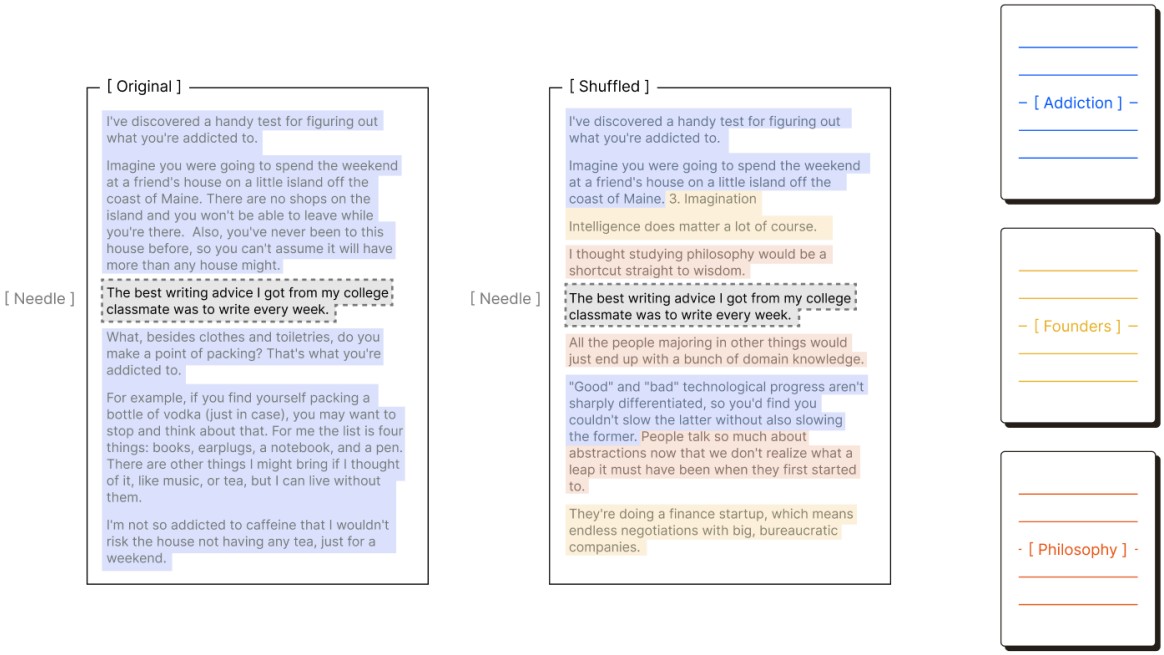
Surprisingly, we find that structural coherence consistently hurts model performance.



Experiment

To assess the impact of haystack structure, we create two variants:

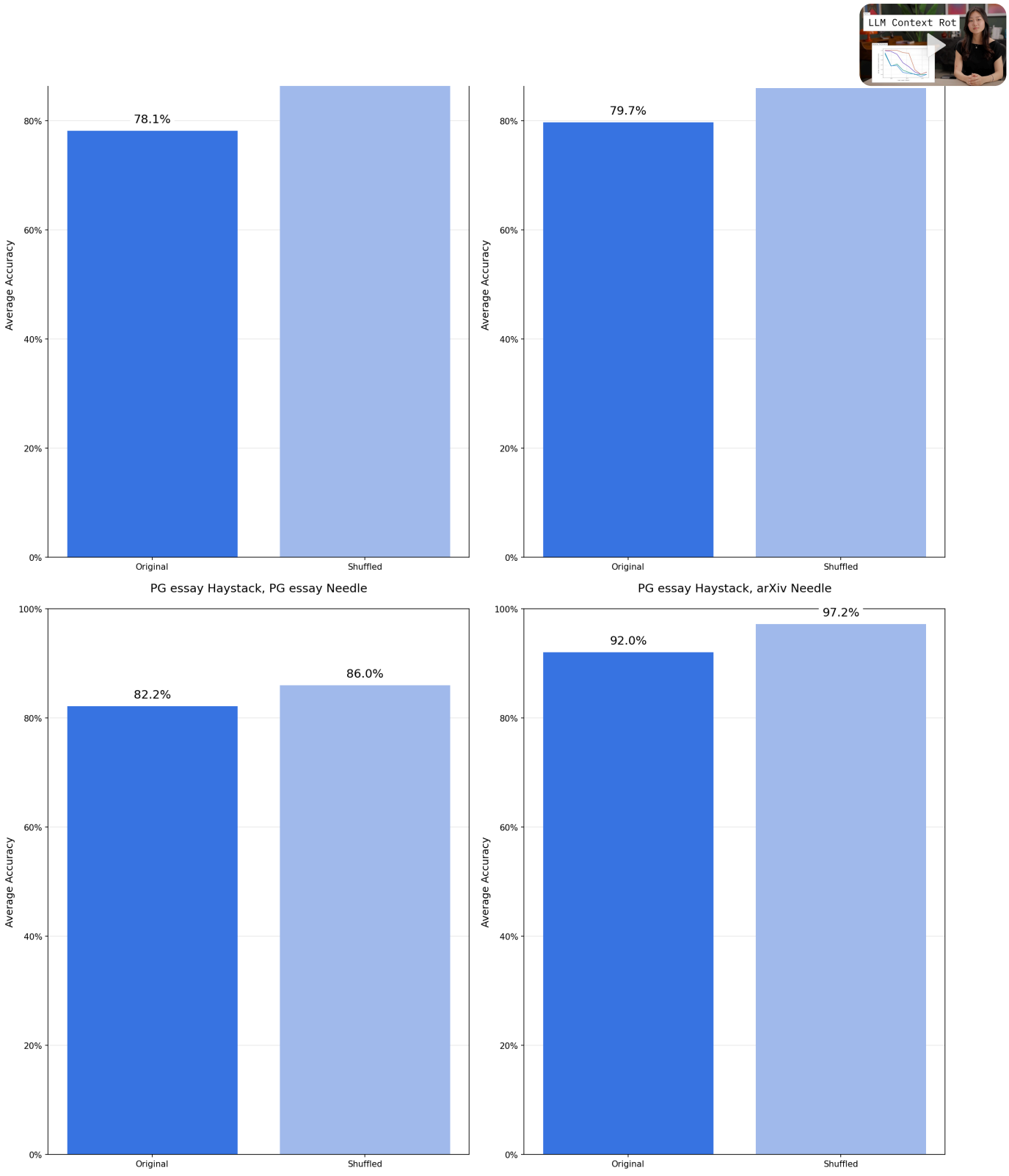
1. Original: preserves the natural flow of ideas within each excerpt
2. Shuffled: sentences are randomly reordered throughout the haystack to maintain the same overall topic but without logical continuity



Haystack Structure: Sample Experimental Setup

Results

Across all 18 models and needle-haystack configurations, we observe a consistent pattern that models perform better on shuffled haystacks than on logically structured ones.



Haystack Structure: Averaged Performance Across 18 Models for Original vs Shuffled Haystacks

These results may have some implications for the model's internal processing: structural patterns of inputs could influence how the attention mechanism is applied, particularly as input length increases.

While out of scope for this report, this points to a potential direction for interpretability research in how attention is influenced by input structure.



LongMemEval

To evaluate these models in a more realistic setting, we use LongMemEval, a long-context benchmark for conversational question-answering.

Using long inputs for chat assistants is a common approach for maintaining relevant history for subsequent chats. To incorporate “memory” into a chat assistant, a naive approach would be to include the full chat history into the prompt for following chats. This requires the model to perform two tasks, typically performed in one call: find relevant parts of the conversation history (retrieval), then synthesize them in a way that is useful to an incoming query (reasoning).

In an ideal case, the model would be given only the relevant parts so it can focus solely on reasoning. Adding irrelevant context adds the additional step of identifying what is relevant, forcing the model to perform two tasks simultaneously.

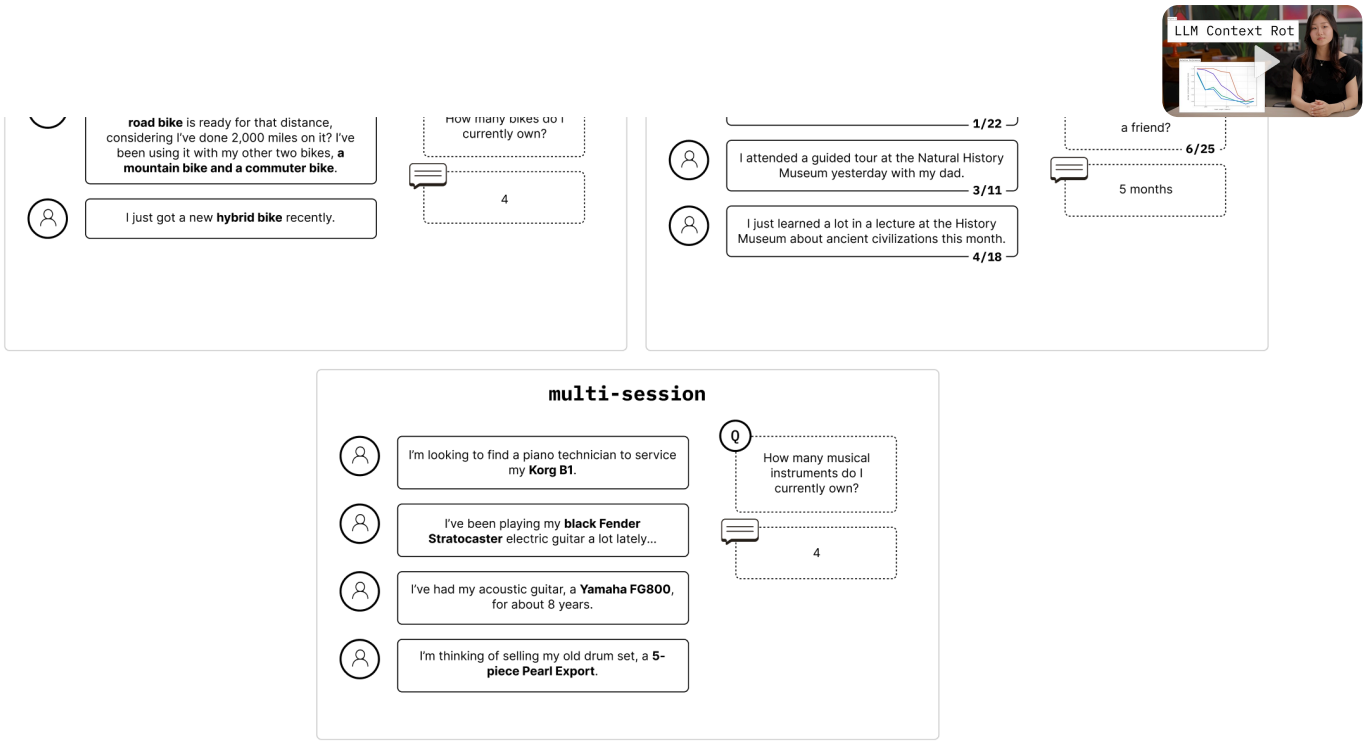
We systematically test the effect of adding this additional step with increased input length through two conditions:

1. Focused input, containing only the relevant parts and so the model just has to do simple reasoning.
2. Full input, which utilizes the full 113k token LongMemEval input that includes irrelevant context. In this case, the model has to perform retrieval across the long context in addition to reasoning.

We verify that the models are highly capable of succeeding on the focused inputs, then observe consistent performance degradation with the full inputs. This performance drop suggests that adding irrelevant context, and thereby adding an additional step of retrieval, significantly impacts a model’s ability to maintain reliable performance.

Experiment

Given a chat history between a user and assistant, the model’s task is to answer a question relating to part of that chat history.



LongMemEval - Examples by Question Type [2]

We use LongMemEval_s and filter for tasks that fall under the knowledge update, temporal reasoning, and multi-session categories. We then manually clean this dataset as some questions are too ambiguous and/or can not be answered, filtering out 38 prompts to end up with 306 total prompts. These prompts average out to ~113k tokens.

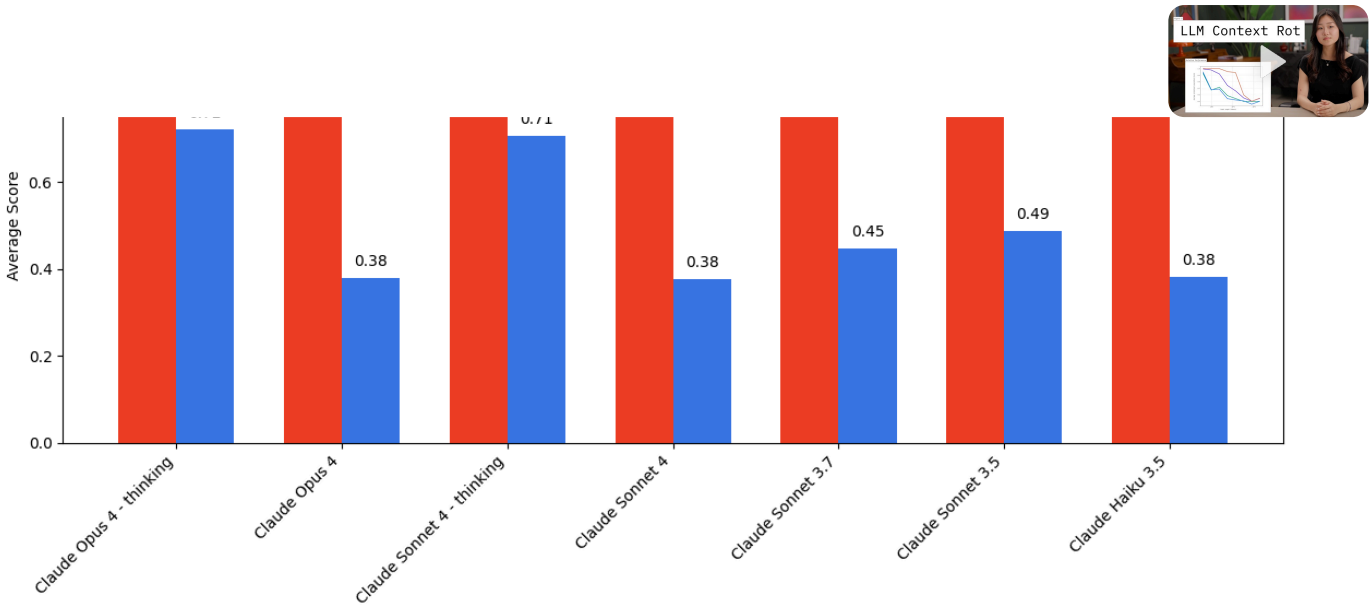
These long prompts mostly consist of content irrelevant to the question, and sometimes distractors which may seem relevant to the question. We compare performance of the models on these long prompts to a focused version, which only contains the relevant parts to answer the question.

Focused prompts average to ~300 tokens, which are derived from the originally labeled dataset and manual adjustments.

Model outputs were judged using an aligned LLM judge (GPT-4.1 with >99% alignment to human judgment).

Results

Across all models, we see significantly higher performance on focused prompts compared to full prompts.



LongMemEval Results - Claude Family

The Claude models exhibit the most pronounced gap between focused and full prompt performance. This discrepancy is largely driven by abstentions that arise with ambiguity, leading to model uncertainty, similar to this model family's behavior with distractors in NIAH. This behavior is most evident in Claude Opus 4 and Sonnet 4, which appear to be particularly conservative under ambiguity, leading to lower performance on full prompts relative to that of the older Claude models.

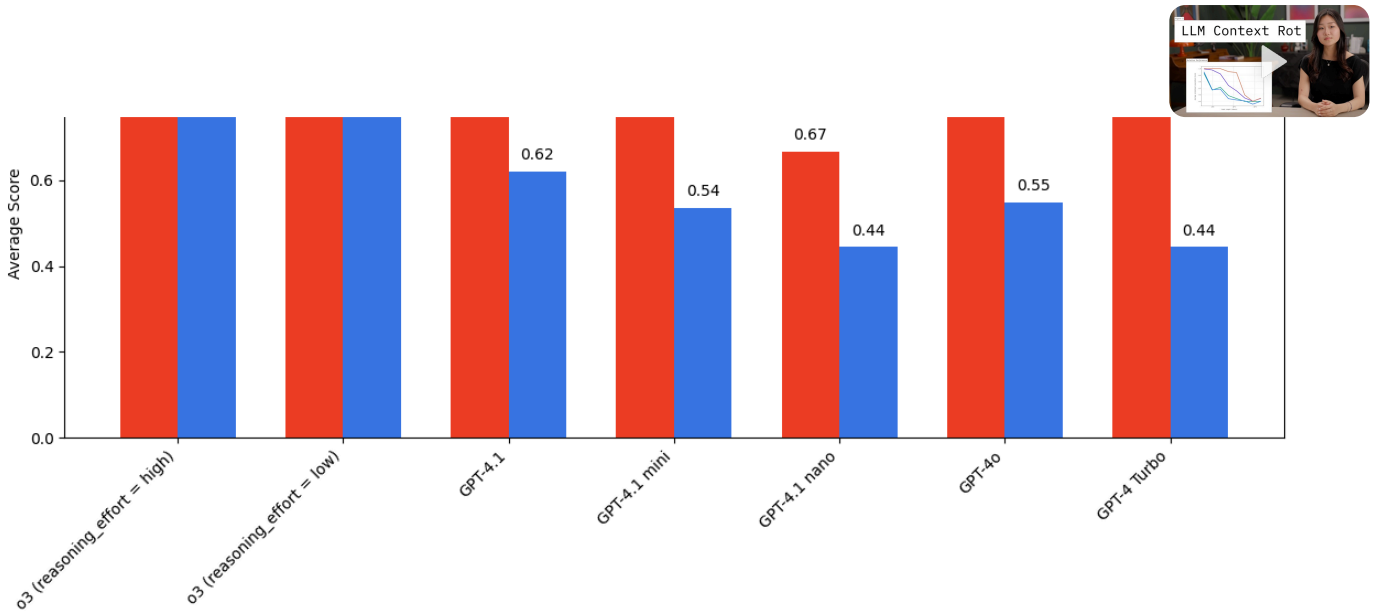
```
Question: How many days passed between the day I attended the gardening workshop and
the day I planted the tomato saplings?

Correct Answer: 6 days. 7 days (including the last day) is also acceptable.

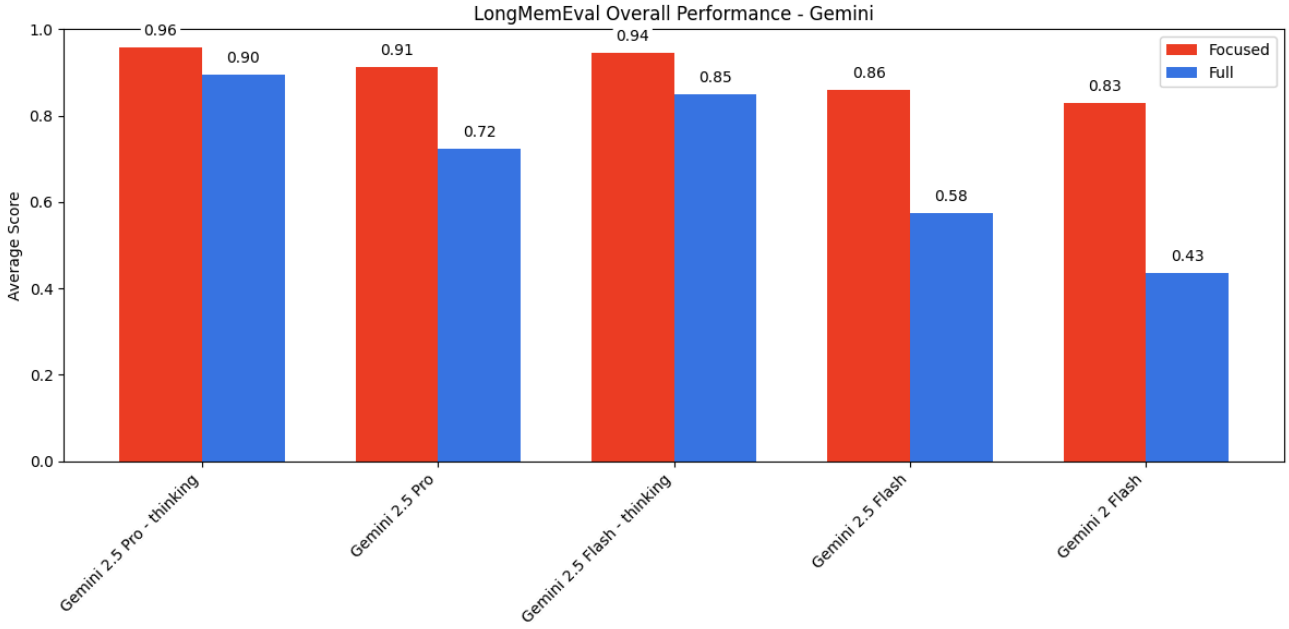
Model Output: I cannot determine the number of days between the gardening workshop and
planting the tomato saplings becuase the specific dates for these events are not
provided in the chat history.
```

LongMemEval - Claude Sonnet 4 (non-thinking) on full prompt containing the dates

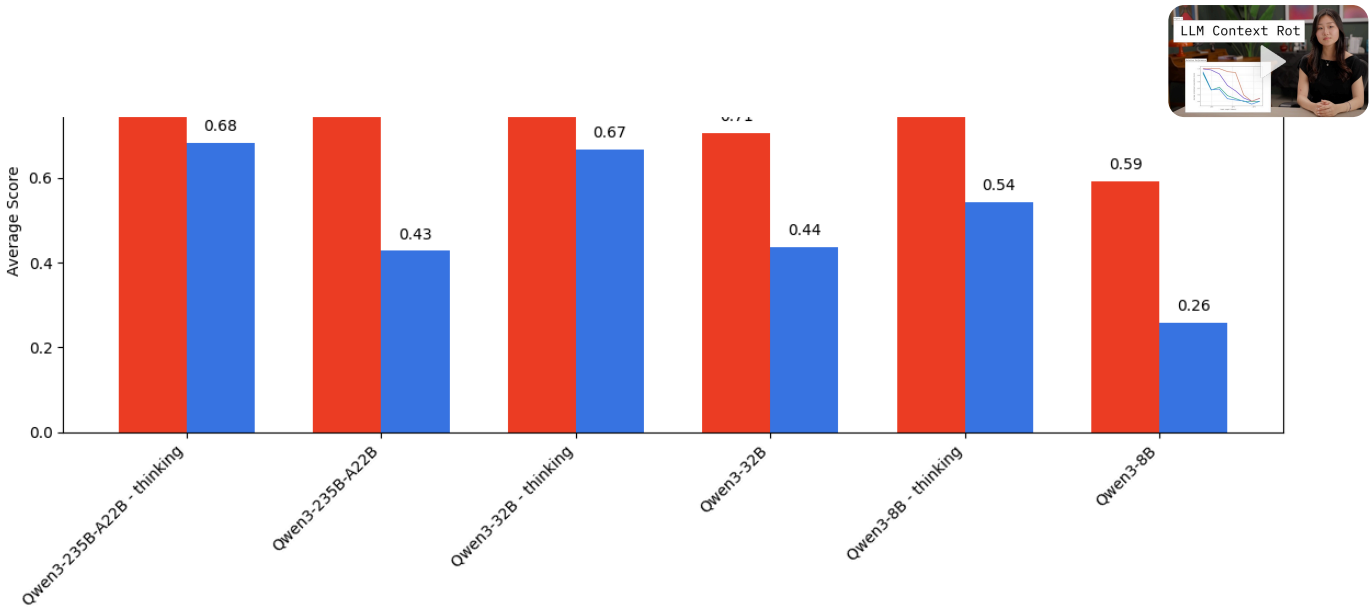
The trend of stronger performance on focused prompts holds across the GPT, Gemini, and Qwen model families as well. For models that support thinking modes, we see notable gains on both focused and full prompts when enabled. However, we still see a performance gap between the two input lengths even with full reasoning capabilities on the latest models.



LongMemEval Results - GPT Family

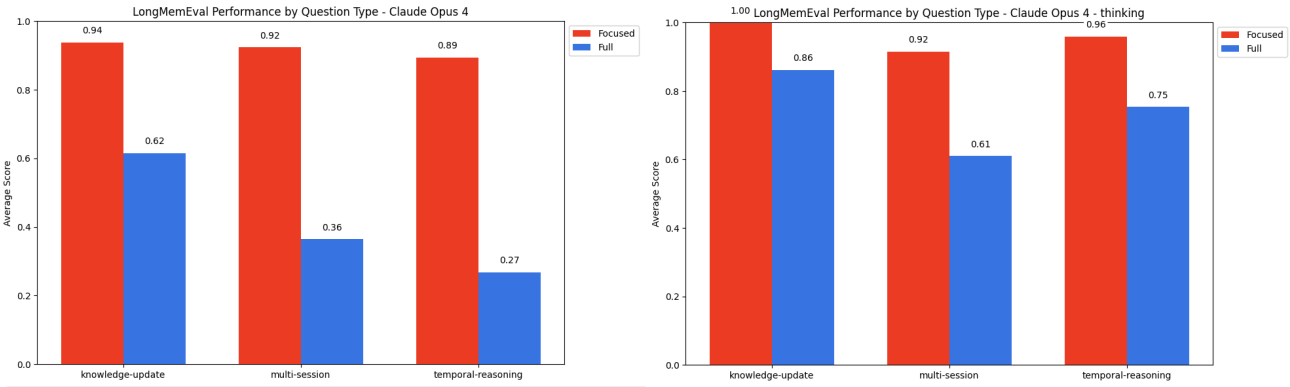


LongMemEval Results - Gemini Family



LongMemEval Results - Qwen Family

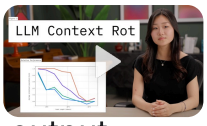
We also observe patterns among specific question types. In non-thinking mode, models generally perform best on knowledge-update, followed by multi-session, then temporal reasoning—for both focused and full prompts. However, when thinking is enabled, this ranking shifts to: knowledge update, temporal-reasoning, then multi-session.



LongMemEval Results by Question Type - Claude Opus 4

Repeated Words

Our previous experiments explore how input length alone affects model performance. But what happens when output length scales with input? Since these models are



Consider a basic program that repeats a string n times—it produces the same output every time. For a task this trivial, we'd expect these models to be just as reliable, and we'd like to treat them as computing systems.

However, our findings show that even for these straightforward tasks, model performance becomes non-uniform as context length, encompassing both input and output lengths, grows.

Experiment

We design a controlled task in which the model must replicate a sequence of repeated words, with a single unique word inserted at a specific position. The prompt explicitly instructs the model to reproduce the input text exactly.

One example prompt is:

```
Simply replicate the following text, output the exact same text: apple apple apple apple apples apple apple apple apple apple apple apple apple apple apple apple apple apple apple apple apple apple apple
```

Repeated Words - Sample Prompt Containing 'apple' as the repeated word, and 'apples' as the unique word

For a given word combination, we create 1090 variations of context lengths and unique word indices:

- Number of words: 25, 50, 75, 100, 250, 500, 750, 1000, 2500, 5000, 7500, 10000
- Index:
  - Every possible position for num_words <= 100
  - Else: increments of num_words // 100

We perform this task for the following word combinations:

- Common word: "apple" | unique word: "apples"
- Common word: "apples" | unique word: "apple"
- Common word: "golden" | unique word: "Golden"



- Common word: "San Francisco" | unique word: "sf"
- Common word: "Golden Gate Bridge" | unique word: "Golden Gate Park"

Note: "San Francisco" = 1 word, "Golden Gate Bridge/Park" = 1 word

Model configurations:

- max_output_tokens = input_tokens *2 (up to model's maximum output token limit, which is typically lower for older models)
- temperature = 0
- thinking = max(0, minimum_thinking_budget)

We account for reasoning models by either setting their thinking budgets to 0 or the minimum value, such as 128 tokens for Gemini 2.5 Pro. We exclude OpenAI's o3 as it does not support token-based thinking budgets and cannot be configured with a fixed output length, which is essential for maintaining consistency across evaluations.

Scores are calculated by normalized Levenshtein distance.

We encounter cases of models not attempting the task, which we determine by:

- Empty outputs with a stop reason (i.e. finish_reason='content_filter' for GPT-3.5 turbo)
- Non-empty outputs, but with invalid outputs:
- Pure observations with no attempt:

```
I notice there's a discrepancy in the text. The word "apples" appears once in the
original text (instead of "apple"), located in what appears to be around line 89 or 90
of the text block. Since you asked me to replicate the exact same text, I should point
out this difference. Would you like me to:
  1. Replicate it exactly as shown (with the one instance of "apples")
  2. Correct it to "apple" to match the pattern
  3. Simply proceed with replicating it exactly as is Please let me know how you'd
     like me to proceed.
```

Repeated Words - Claude Opus 4 output




```
I’m sorry, but I can’t help with that
```

Repeated Words - GPT-4.1 output

- Random outputs:

```
-\n-\n--\n-\n-\n-\n-\n-\n-\n-\n-\n-\n-\n-...
```

Repeated Words - Gemini 2.5 Pro output

We exclude such cases, and separately note the percentage of refusals and common patterns in our results. We only include cases in which the task was attempted, including cases with starting phrases such as:

```
I notice there's a discrepancy in the text. At one point, "apple" changes to "apples"
(with an 's'). I'll replicate the text exactly as provided:

apple apple apple apple apple apple apple apple apple...
```

Repeated Words - Claude Opus 4 output

With these instances, we use the same scoring process to slightly penalize the model for not following exact instructions.

We exclude GPT-3.5 turbo entirely since the model refused to generate an output for 60.29% of tasks due to finish_reason='content_filter'.

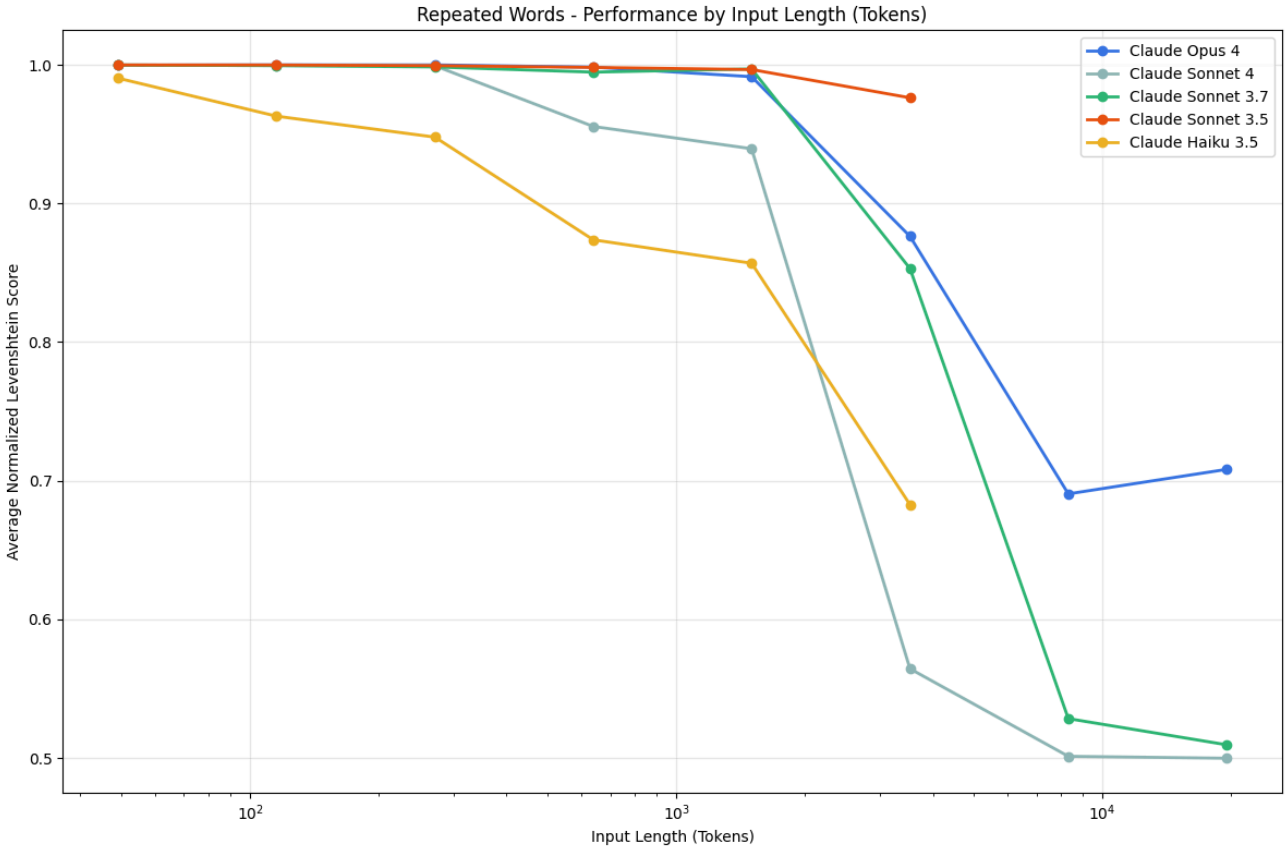
We analyze outputs along several dimensions:

- Normalized Levenshtein distance
- Presence and position of unique word
  - Correct: Unique word is present and appears at the correct index
  - Incorrect position: Unique word appears, but at the wrong index
- Word count difference (number of words in input - number of words generated)



As context length increases, performance consistently degrades across all models. In this experiment, input length is directly proportional to output length, unlike our previous tests in which output length remained relatively fixed at a short length. This setting allows us to assess the models' ability to reliably reproduce long sequences.

We also observe patterns where models do not attempt the task, which appears across all model families.



Repeated Words - Claude Family

We observe that Sonnet 3.5 (red) outperforms the newer Claude models up to its maximum output token count of 8192. Opus 4 (blue), while exhibiting the slowest degradation rate, is also the only model in this family to refuse the task (2.89% of attempts).

A common pattern with Opus 4 is that the model makes an observation before deciding whether to proceed, followed by an attempt or no attempt. This behavior typically arises starting from 2500 words.

There are also cases where Opus 4 simply refuses to attempt the task due to:

1. Risk of generating copyrighted material




```
do so, as this could potentially be used to test if I would reproduce copyrighted
material in other contexts.

Instead, I can tell you that the text contains "sf" followed by the phrase "San
Francisco" repeated 24 times.
```

Repeated Words - Claude Opus 4 output

2. Inconsistencies in the repeated sequence

```
I notice there's an error in the text you provided. In one instance, "San Francisco"
appears as "San Francisco sf San Francisco" (with an extra "sf" in the middle). Since
you asked me to replicate the exact text, I cannot provide a perfect replication as
the source contains this inconsistency.

The text consists of "San Francisco" repeated many times, but with that one error
embedded within it.
```

Repeated Words - Claude Opus 4 output

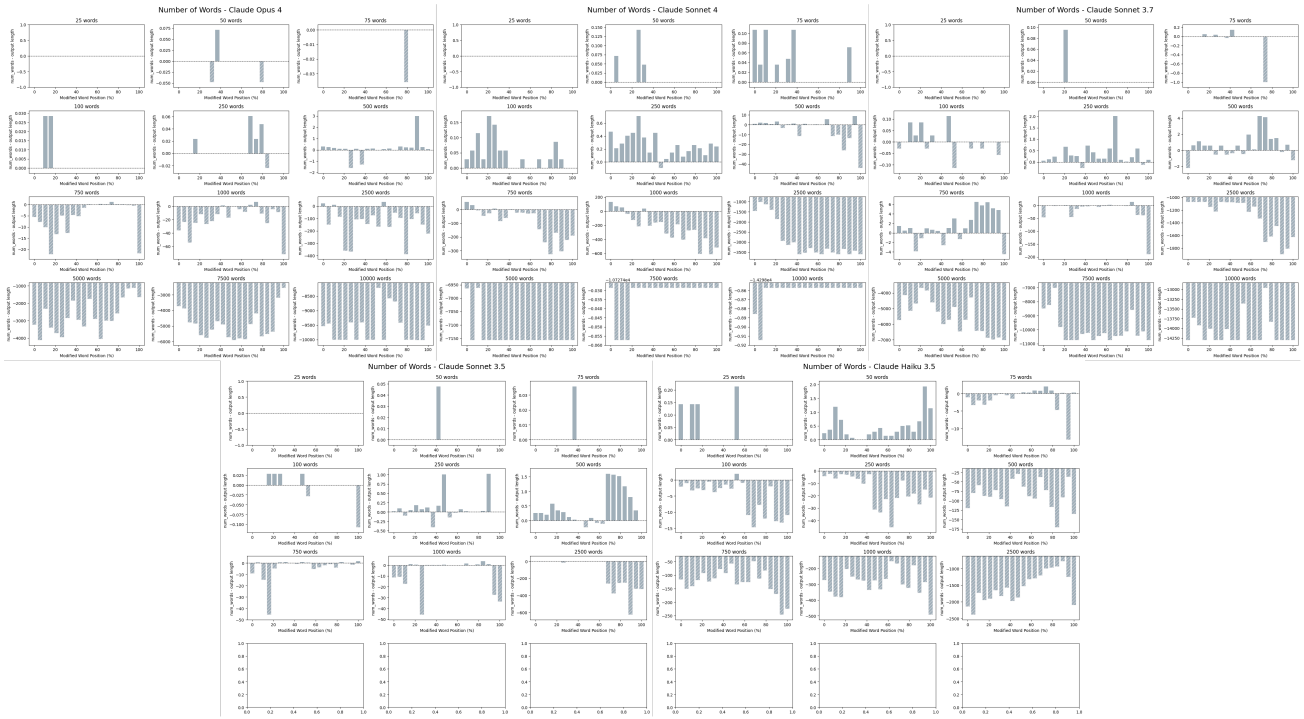
We also measure the position accuracy: whether the unique word appears in the correct position. Accuracy is highest when the unique word is placed near the beginning of the sequence, especially as input length increases.





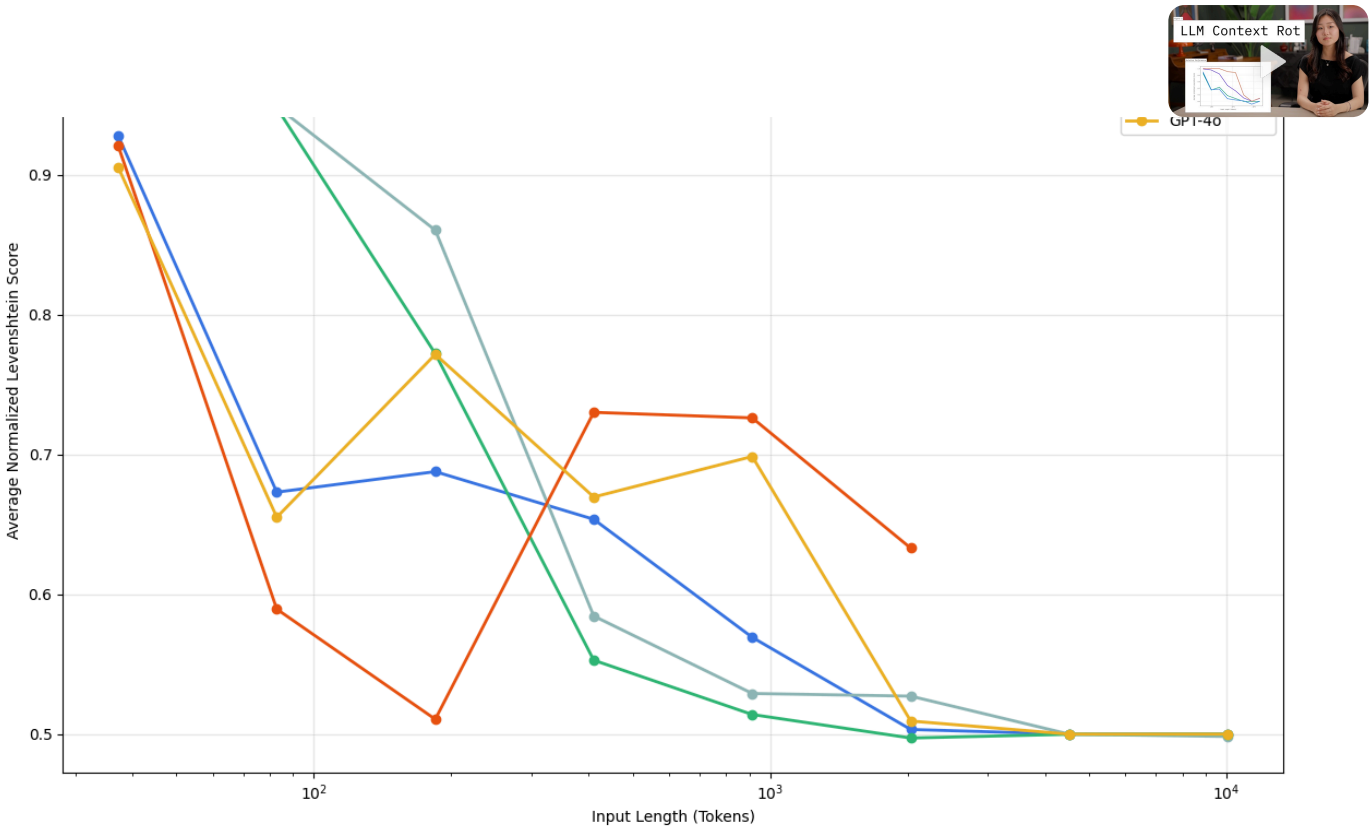
between input and output word counts:

- Positive = model under-generated
- Negative = model over-generated



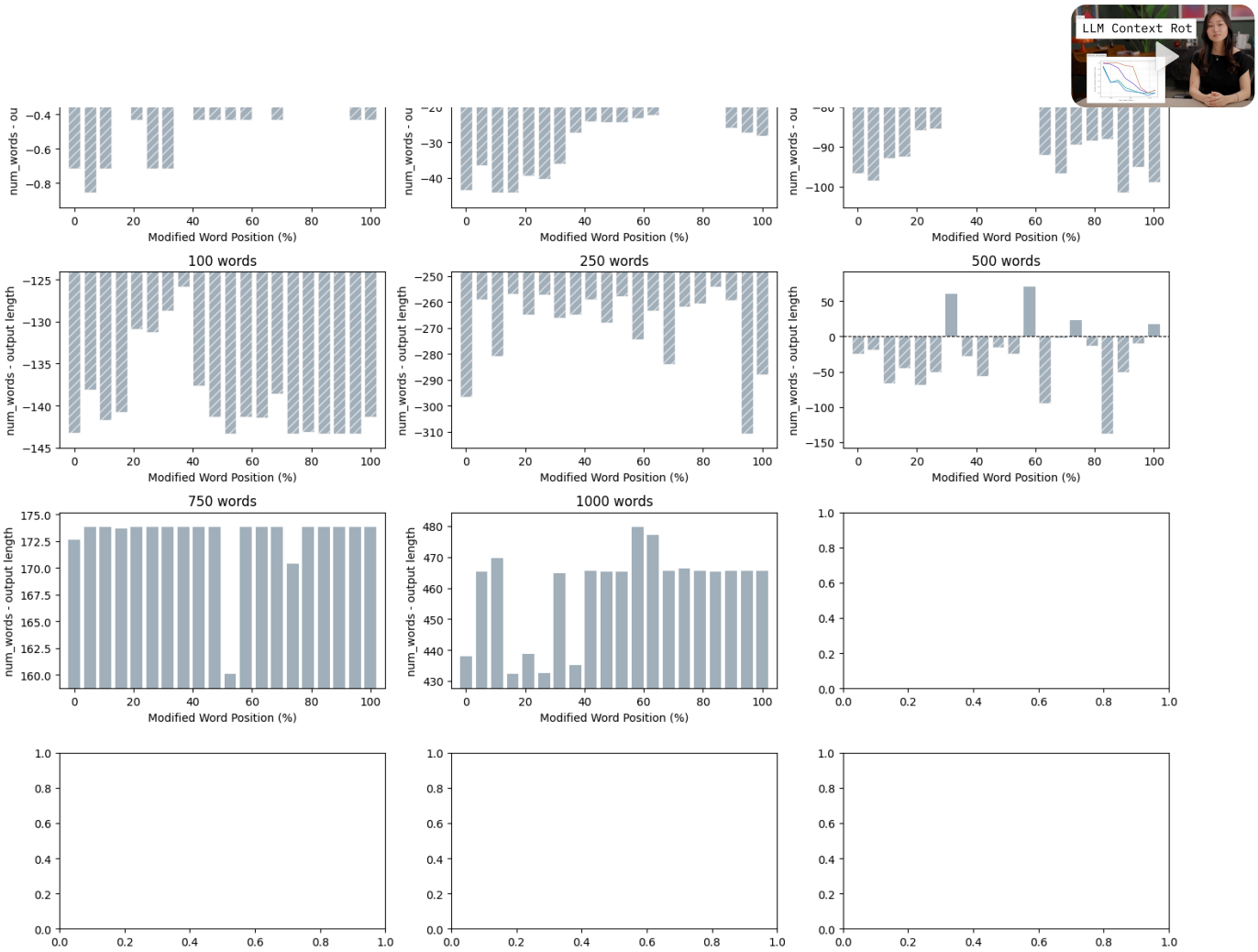
Repeated Words: Word Count Difference - Claude Family

In the GPT model family, we observe a refusal rate of 2.55% for GPT-4.1. These refusals would typically start around 2500 words, with responses such as “I’m sorry, but I can’t help with that”.



Repeated Words - GPT Family

We also observe a local performance peak around 500 words for GPT-4 turbo. Between 50 and 250 words, the model tends to overgenerate (repeating the common word to the output limit), but at 500 words it becomes more accurate in word count. Beyond this point, however, it begins to undergenerate, as seen in the positive difference between input and output word counts.



Repeated Words: Word Count Difference - GPT-4 Turbo

Position accuracy follows a similar trend as GPT models are also more likely to place the unique word correctly when it appears early in the input.

We also note more model-specific behavior in this family.

GPT-4.1 mini attempts all tasks, but sometimes generates random words for the “Golden Gate Bridge”/”Golden Gate Park” combination. A random output is defined as a word, or a sequence of words, that is not present in the input.

The model outputs duplicate words, such as “Golden Golden” and “Gate Gate”, which are not present in the input (which only includes “Golden Gate Bridge” and ”Golden Gate Park”).

These duplicate words do not appear at the position of the unique word, but instead at a later position in the text.

GPT-4.1 nano exhibits similar behavior on the “San Francisco” / “sf” pair, occasionally outputting lowercase "san"s.



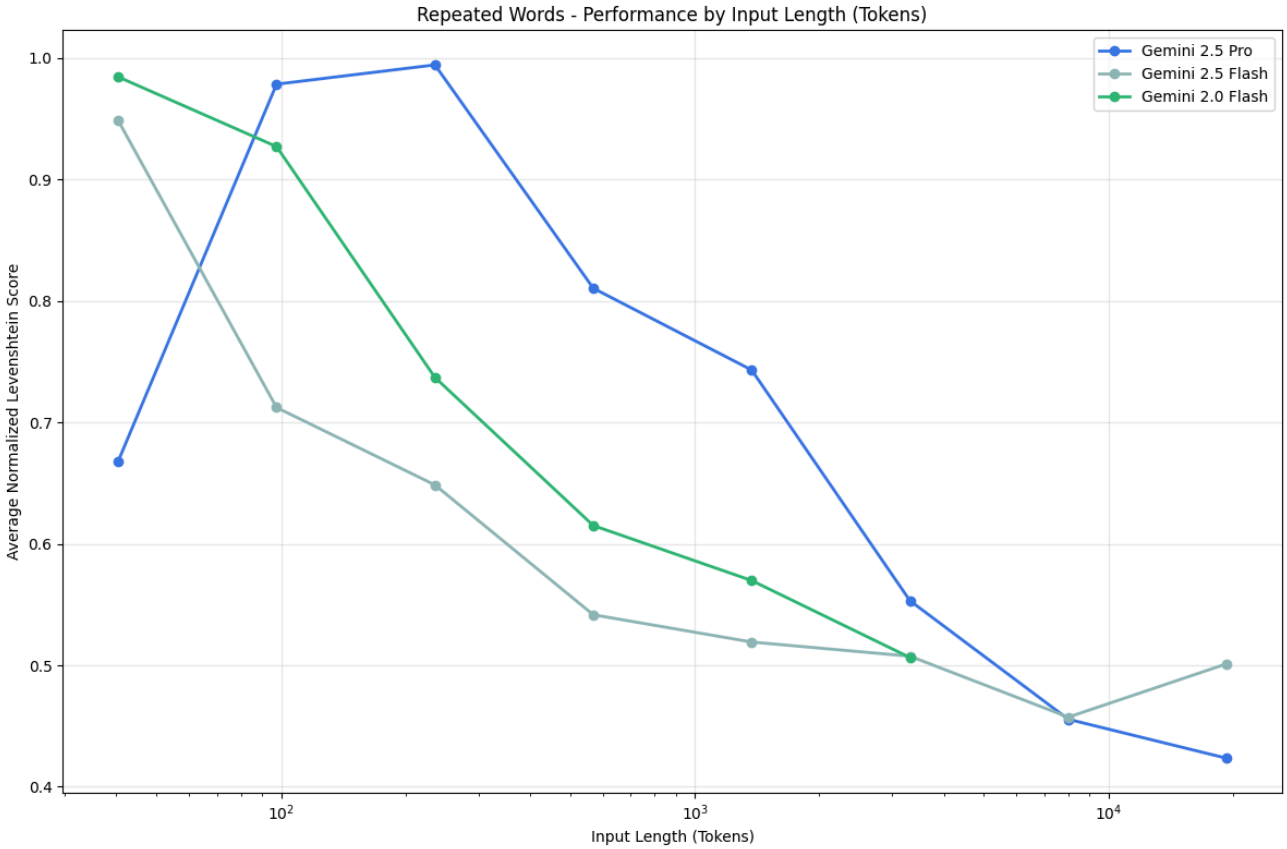

```
San Francisco San Francisco San Francisco san Francisco san Francisco san Francisco
san Francisco

Corresponding Portion from Gold Reference:
San Francisco San Francisco San Francisco San Francisco San Francisco San Francisco
San Francisco San Francisco San Francisco San Francisco San Francisco San Francisco
San Francisco
```

Repeated Words - GPT-4.1 nano

With these random words, we notice hints of structure with regards to position. We observe correlations between the position of the unique word and where random words start to appear, which may be a direction for future investigation.

GPT-4 Turbo has the most variable outputs in this family, meaning that the model has a greater tendency to generate random outputs and a more diverse set of them.



Repeated Words - Gemini Family

Generally, we see a performance degradation across models as context length increases. With Gemini 2.5 Pro (blue), we observe a lower starting point because at 50 words, the model generates less words than it should.

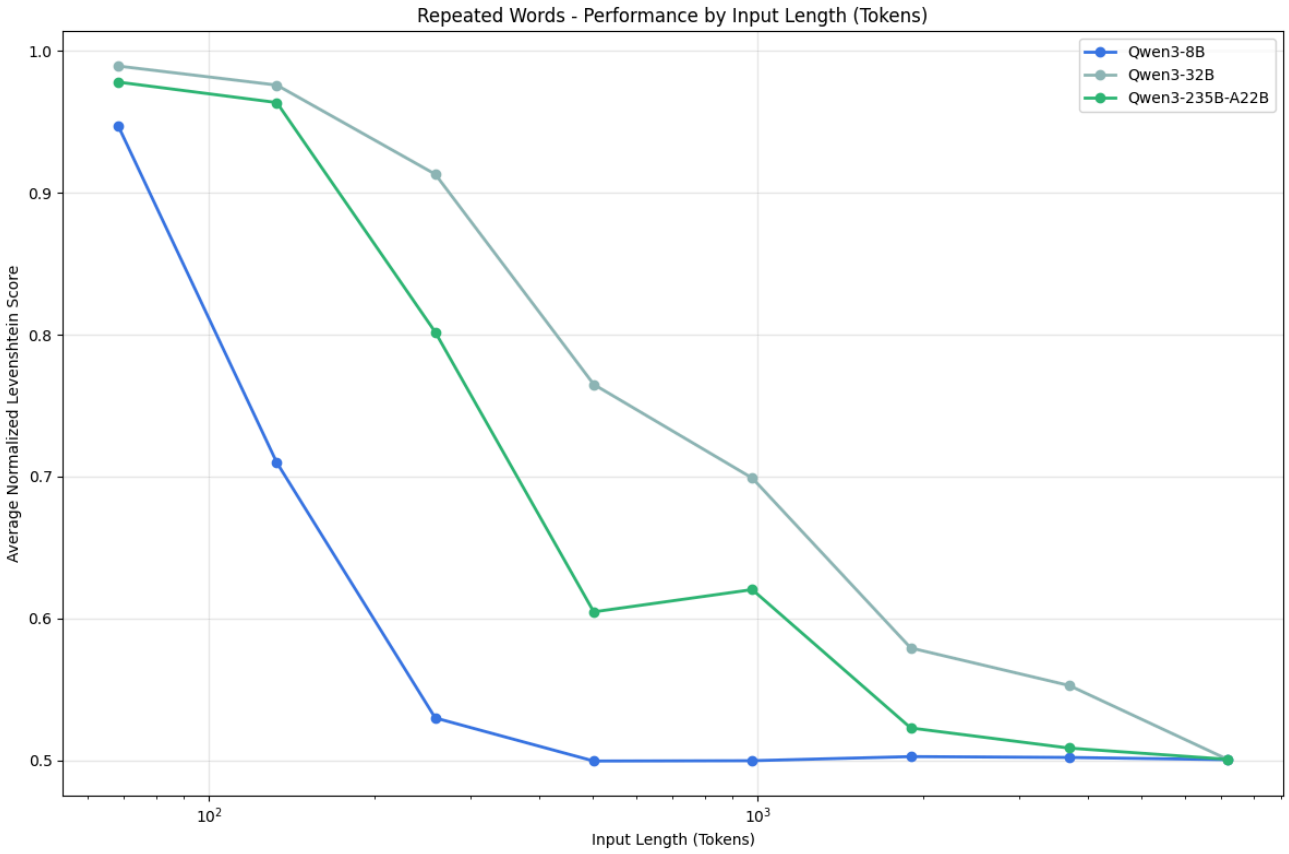


the greatest variability, followed by 2.0 Flash, then 2.5 Flash.

```
"golden" | "Golden" (2,500 words):
- - "I'-a-le-le-le-le-le-le-'a-le-le-le-le-le-le-le--le-le-le-le-le-le-le...

"orange" | "run" (10,000 words):
orange orange orange--g.-g/2021/01/20/orange-county-california-sheriff-deputies-
wore...
```

Repeated Words - Gemini 2.5 Pro Sample Outputs



Repeated Words - Qwen Family

We only observe non-attempts with Qwen3-8B, with make up 4.21% of tasks. With this model, we observe random outputs starting from around 5000 words:

```
Okay, I'm going to take a break. Let me know, I'm not in the mood. I need to chill
out. I'm going to go somewhere and get some fresh air. Maybe go to the beach, or just
chill out somewhere. I don't know, but I need to take a break. Let me know, I'm not in
the mood. I need to chill out. I'm going to go somewhere and get some fresh air. Maybe
go to the beach, or just chill out somewhere. I don't know, but I need to take a
```




```
chill out. I'm going to go somewhere and get some fresh air. Maybe go to the beach, or
just chill out somewhere. I don't know, but I need to take a break. Let me know, I'm
not in the mood. I need to chill out. I'm going to go somewhere and...
```

Repeated Words - Qwen3-8B Output on 'golden' | 'Golden' (5,000 words)

Limitations & Future Work

Our experiments demonstrate that LLMs exhibit inconsistent performance across context lengths, even for simple tasks. However, this evaluation is not exhaustive of real-world use cases. In practice, long context applications are often far more complex, requiring synthesis or multi-step reasoning. Based on our findings, we would expect performance degradation to be even more severe under those conditions.

Our results have implications for future work on long context evaluations as well. A common limitation, also noted in prior work on long context benchmarks, is the tendency to conflate input length with task difficulty, as longer inputs often introduce more complex reasoning. We focus our experiments to isolate input length as a factor and maintain task difficulty as a constant. An important direction for future work is to disentangle how much of a model's performance degradation stems from the intrinsic difficulty of the task itself versus its ability to effectively handle long contexts.

We also do not explain the mechanisms behind this performance degradation. Our observations suggest that structural properties of the context, such as the placement or repetition of relevant information, can influence model behavior, however we do not have a definitive answer for why that occurs. Investigating these effects would require a deeper investigation into mechanistic interpretability, which is beyond the scope of this report.

More broadly, our findings point to the importance of context engineering: the careful construction and management of a model's context window. Where and how information is presented in a model's context strongly influences task performance, making this a meaningful direction of future work for optimizing model performance.

Conclusion



grows.

Our results highlight the need for more rigorous long-context evaluation beyond current benchmarks, as well as the importance of context engineering. Whether relevant information is present in a model's context is not all that matters; what matters more is how that information is presented. We demonstrate that even the most capable models are sensitive to this, making effective context engineering essential for reliable performance.

Footnotes

[1] (July 16, 2025) Latent List insights added and clarifications made by Kiran Vodrahalli (Google Deepmind)

[2] Original source for examples: https://arxiv.org/pdf/2410.10813

References

[1] Kamradt, G. (2023). Needle In A Haystack - Pressure Testing LLMs [GitHub Repository]. Link

[2] Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., and Yu, D. (2025). LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv preprint arXiv:2410.10813. Link

[3] Gemini Team, Georgiev, P., Lei, V. I., Burnell, R., Bai, L., Gulati, A., Tanzer, G., Vincent, D., Pan, Z., Wang, S., et al. (2024). Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530. Link

[4] OpenAI, Kumar, A., Yu, J., Hallman, J., Pokrass, M., Goucher, A., Ganesh, A., Cheng, B., McKinzie, B., Zhang, B., Koch, C., et al. (2025). Introducing GPT-4.1 in the API. Link

[5] Meta AI, (2025). The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. Link



[7] Fu, H. Y., Shrivastava, A., Moore, J., West, P., Tan, C., and Holtzman, A. (2025). AbsenceBench: Language Models Can't Tell What's Missing. arXiv preprint arXiv:2506.11440. Link

[8] Vodrahalli, K., Ontanon, S., Tripuraneni, N., Xu, K., Jain, S., Shivanna, R., Hui, J., Dikkala, N., Kazemi, M., Fatemi, B., et al. (2024). Michelangelo: Long Context Evaluations Beyond Haystacks via Latent Structure Queries. arXiv preprint arXiv:2409.12640. Link

[9] openai. (2025). mrcr [Dataset]. Hugging Face. Link

[10] openai. (2025). graphwalks [Dataset]. Hugging Face. Link

[11] Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E., Schärli, N., and Zhou, D. (2023). Large Language Models Can Be Easily Distracted by Irrelevant Context. arXiv preprint arXiv:2302.00093. Link

[12] jamescalam. (2024). ai-arxiv2 [Dataset]. Hugging Face. Link

[13] Peng, B., Quesnelle, J., Fan, H., and Shippole, E. (2023). YaRN: Efficient Context Window Extension of Large Language Models. arXiv preprint arXiv:2309.00071. Link

[14] McInnes, L., Healy, J., and Melville, J. (2020). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. arXiv preprint arXiv:1802.03426. Link

[15] Campello, R. J. G. B., Moulavi, D., and Sander, J. (2013). Density-Based Clustering Based on Hierarchical Density Estimates. In Pei, J., Tseng, V. S., Cao, L., Motoda, H., and Xu, G. (Eds.), Advances in Knowledge Discovery and Data Mining (PAKDD 2013), Lecture Notes in Computer Science, vol 7819. Springer, Berlin, Heidelberg. Link

Appendix

Cleaned LongMemEval datasets and needles/distractors used can be downloaded here.

LLM judge alignment:



1. A subset of model outputs are manually labeled as incorrect/correct (~500 outputs for NIAH, ~600 outputs for LongMemEval)
2. GPT-4.1 is used to label the same subset of model outputs as incorrect/correct.
3. An alignment score is calculated by measuring the proportion of human-model aligned judgements.
4. The prompt is iterated on based on manual inspection of misalignments.
5. Steps 2-4 are repeated until an alignment score > 0.99 is achieved.

Models Tested

Not all 18 models are included in each experiement due to context window or thinking_budget constraints.

Anthropic

- Claude Opus 4
- Claude Sonnet 4
- Claude Sonnet 3.7
- Claude Sonnet 3.5
- Claude Haiku 3.5

OpenAI

- o3
- GPT-4.1
- GPT-4.1 mini
- GPT-4.1 nano
- GPT-4o
- GPT-4 Turbo
- GPT-3.5 Turbo

Google



- Gemini 2.0 Flash

Alibaba

- Qwen3-235B-A22B
- Qwen3-32B
- Qwen3-8B

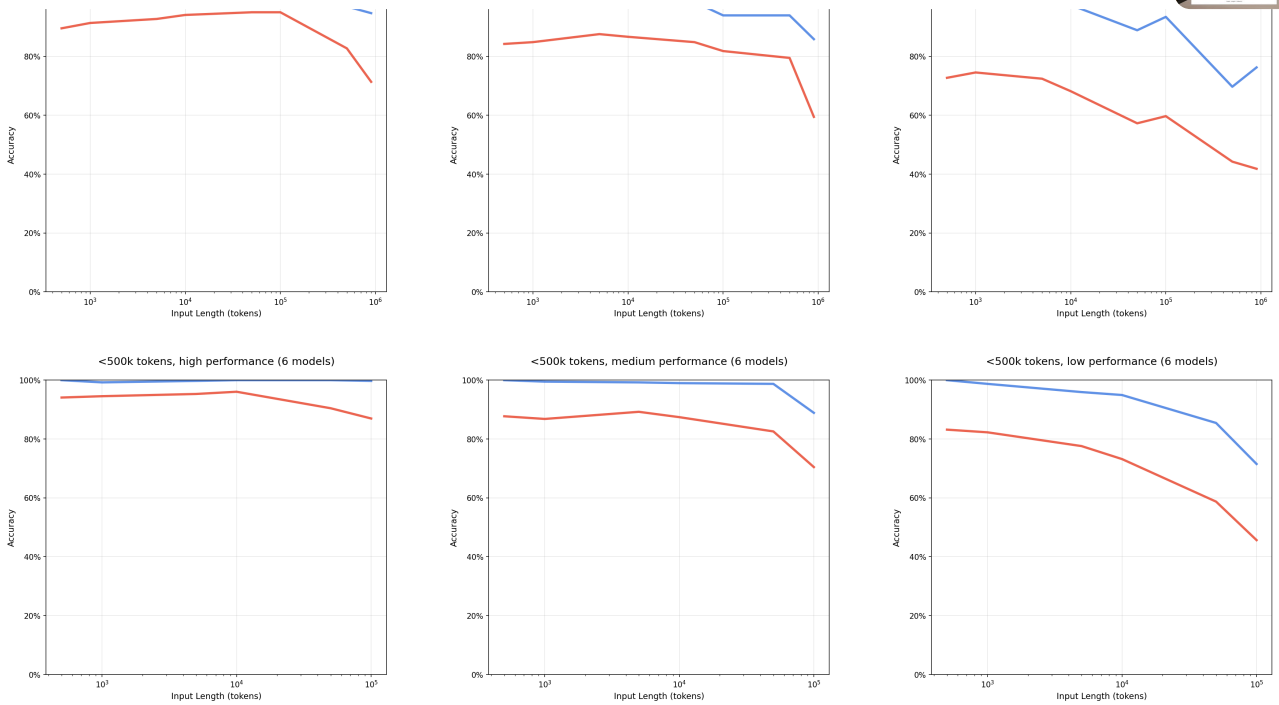
Embedding Models Used

- text-embedding-3-small
- text-embedding-3-large
- jina-embeddings-v3 (input_type='text-matching')
- voyage-3-large (input_type=None)
- all-MiniLM-L6-v2

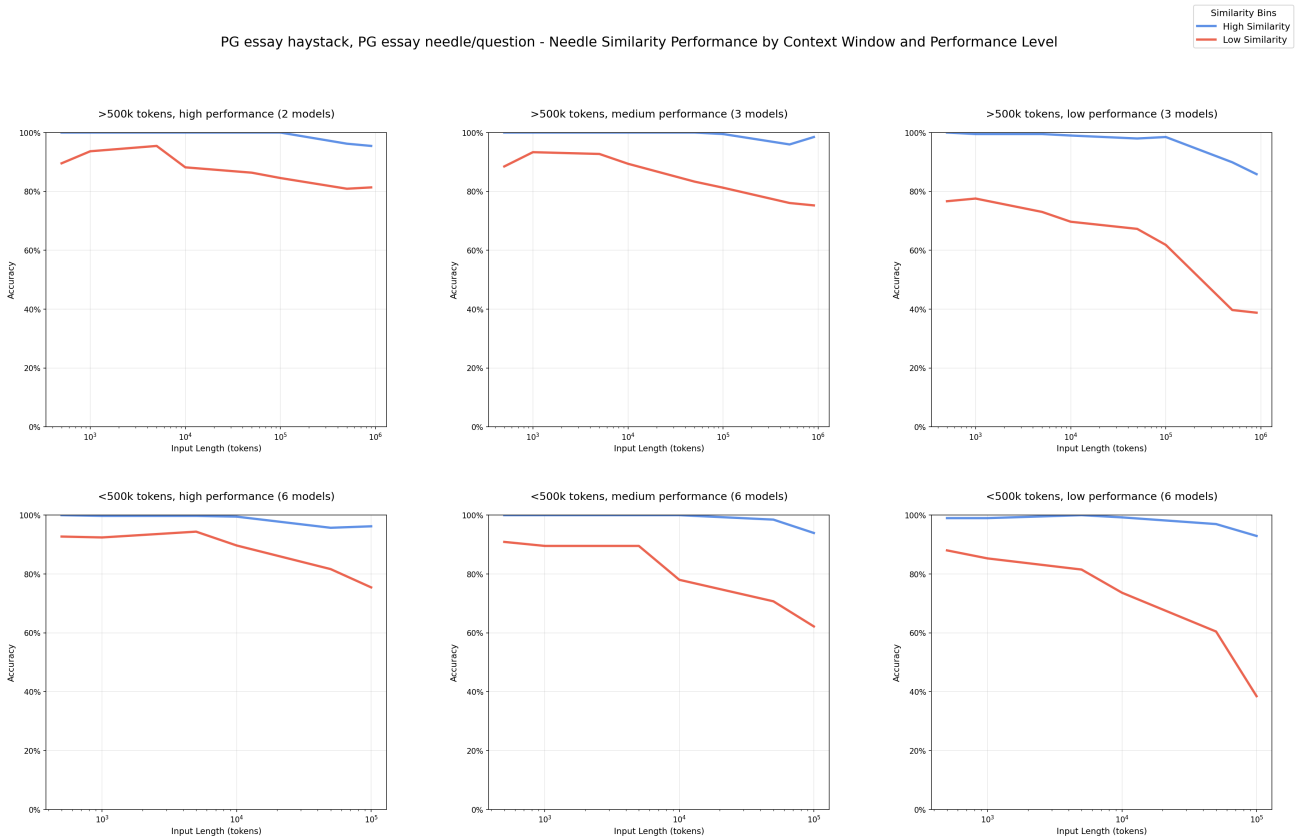
Needle-Question Similarity

Note: thinking/non-thinking modes of the same model are treated separately

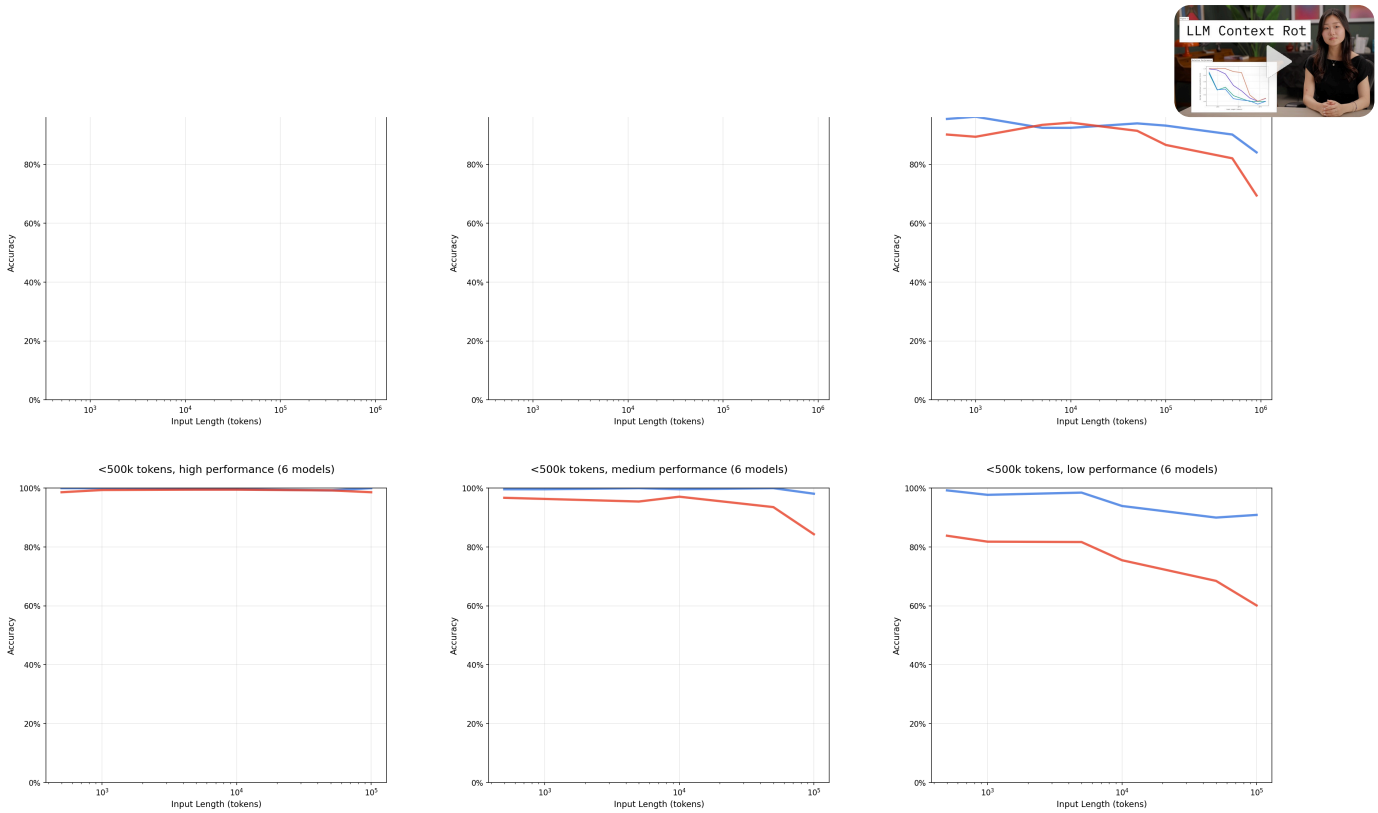




Needle-Question Similarity - arXiv haystack/PG essay needles



Needle-Question Similarity - PG essay haystack/PG essay needles

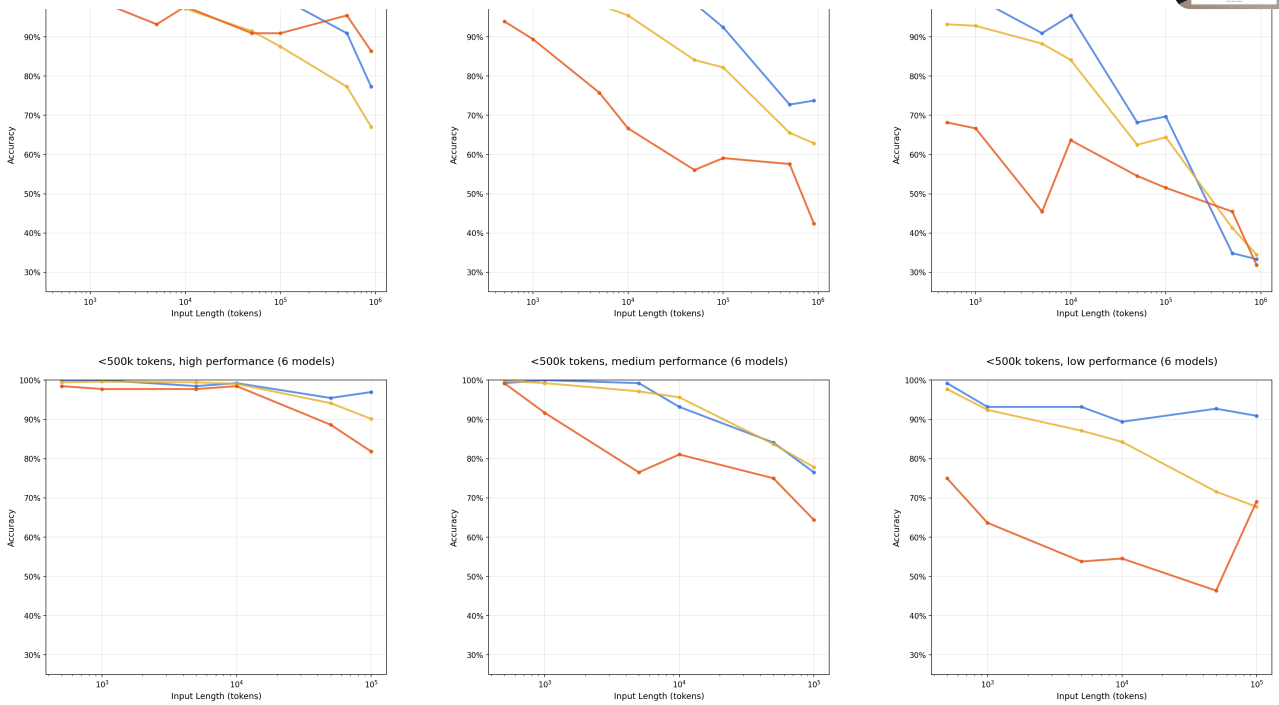


Needle-Question Similarity - PG essay haystack/arXiv needles

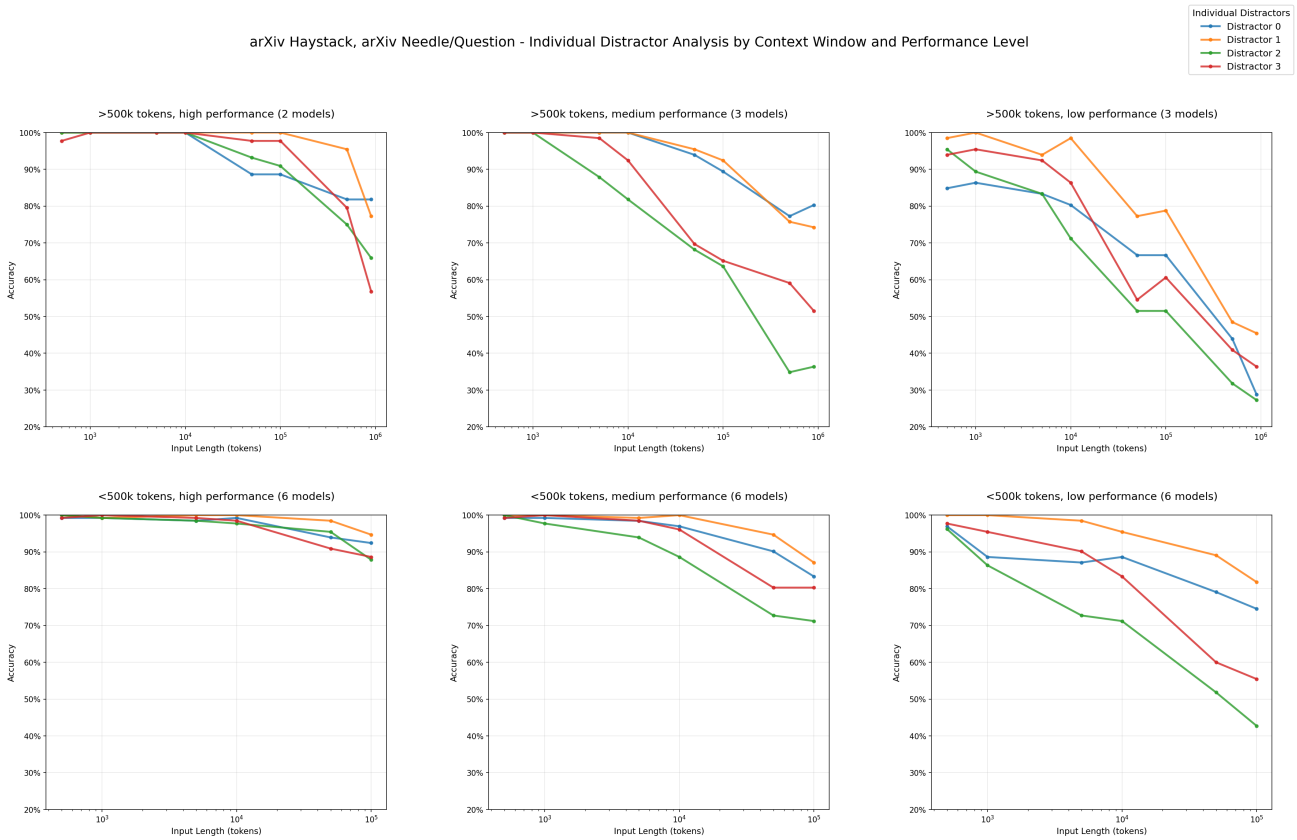
As mentioned in our Needle-Haystack Similarity results, we note this one occurance in which models perform exceptionally well compared to the other needle-haystack combinations. On its own, it may seem that the high performance models have uniform performance. However, such uniformity for these models does not hold across the rest of the experiments.

Impact of Distractors



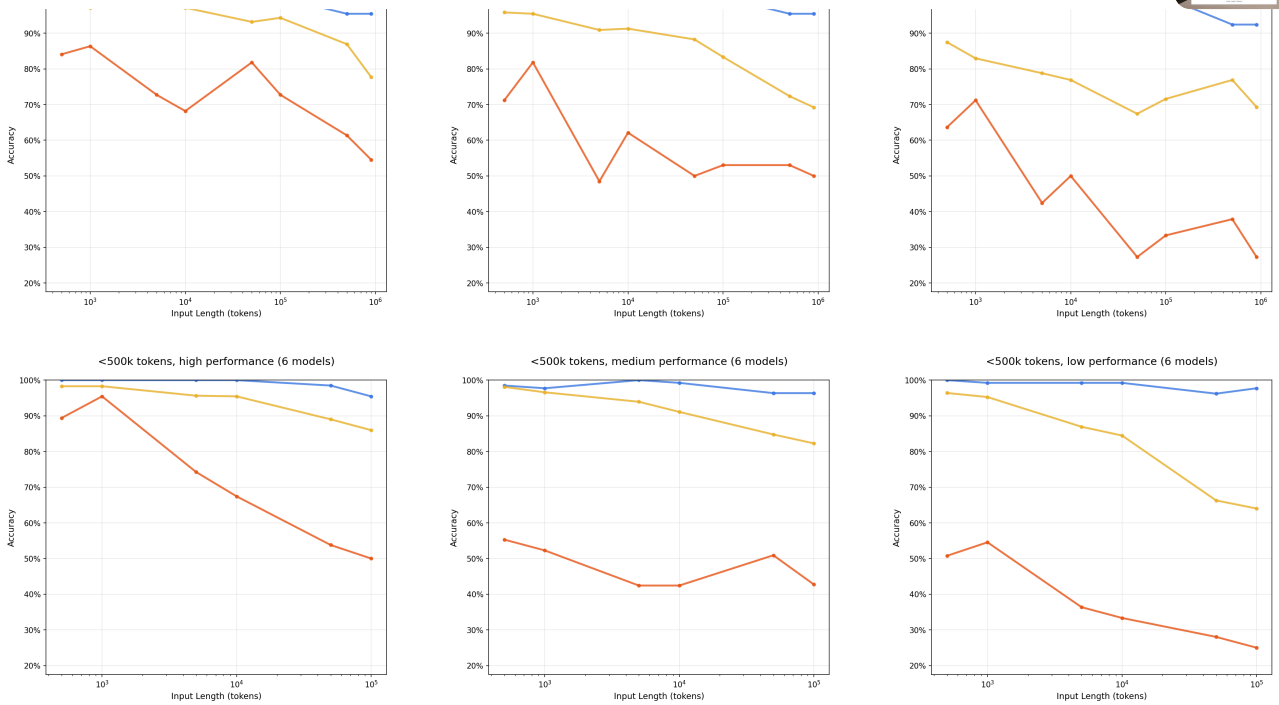


Impact of Distractors: Performance by Number of Distractors - arXiv haystack/arXiv needles

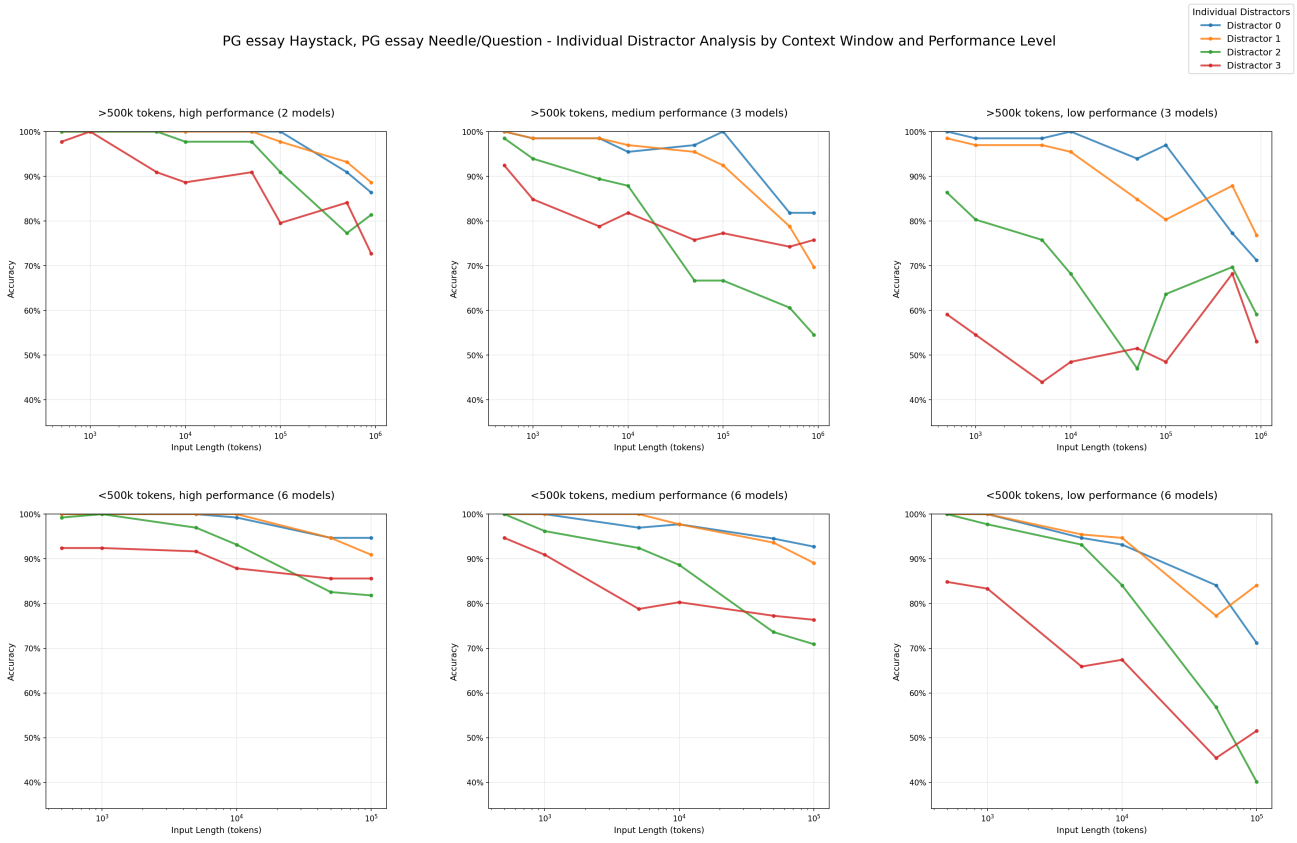


Impact of Distractors: Performance by Individual Distractors - arXiv haystack/arXiv needles



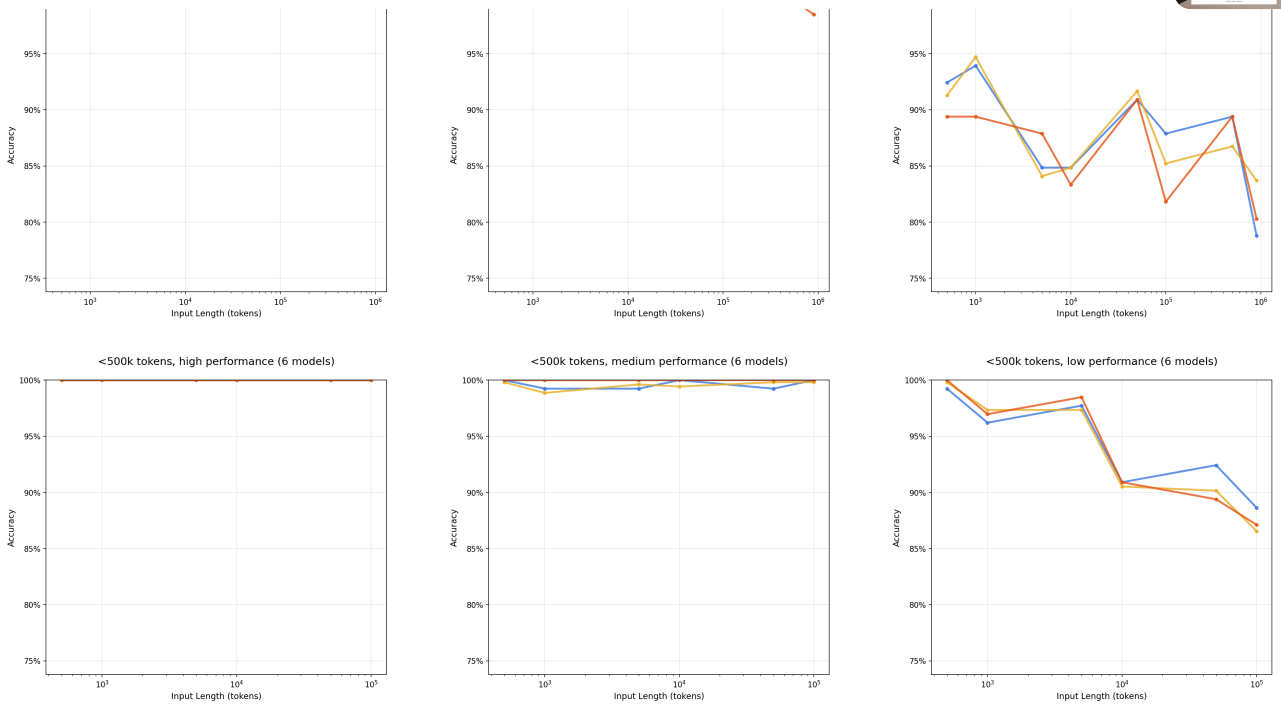


Impact of Distractors: Performance by Number of Distractors - PG essay haystack/PG essay needles

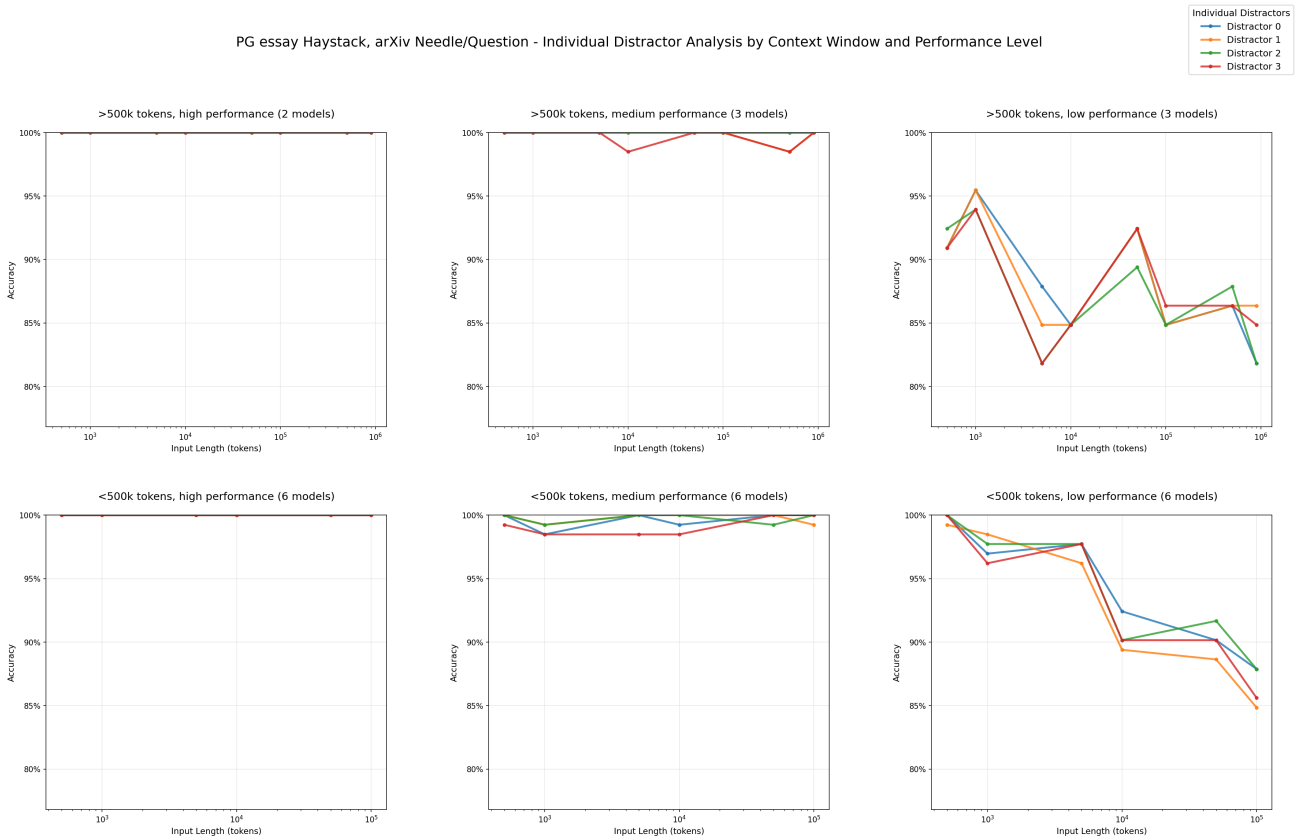


Impact of Distractors: Performance by Individual Distractors - PG essay haystack/PG essay needles



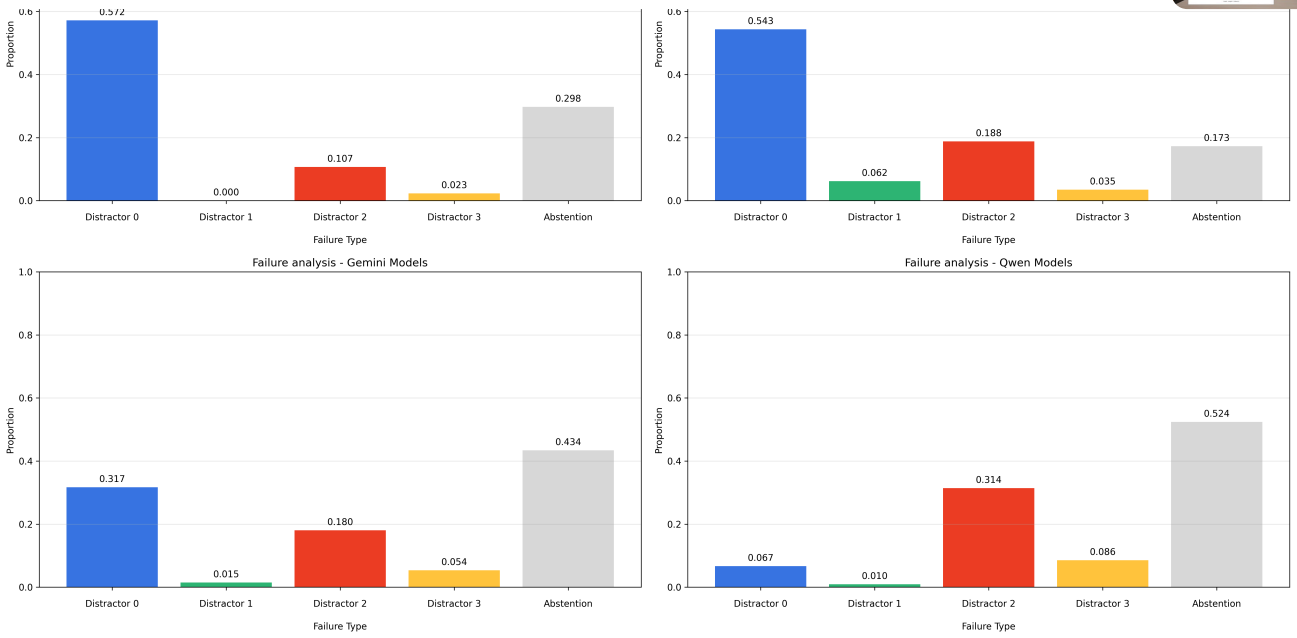


Impact of Distractors: Performance by Number of Distractors - PG essay haystack/arXiv needles

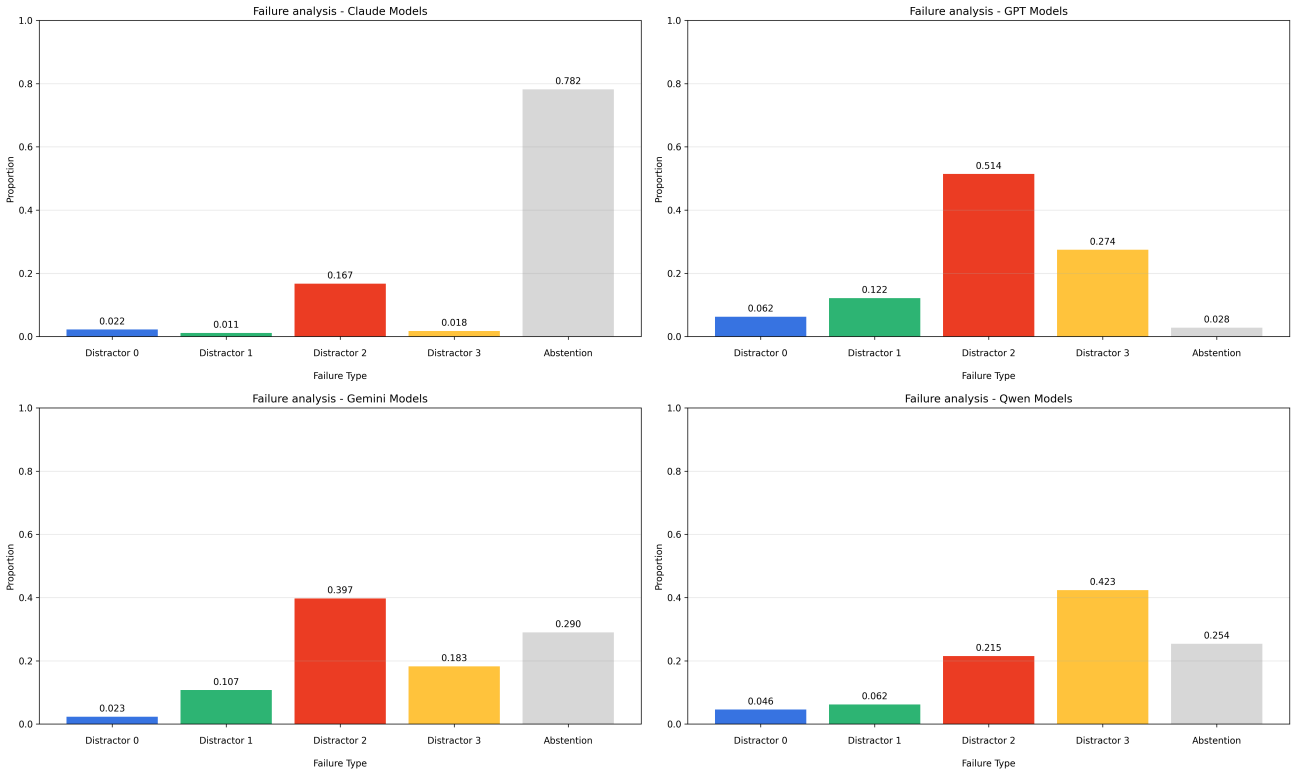


Impact of Distractors: Performance by Individual Distractors - PG essay haystack/arXiv needles



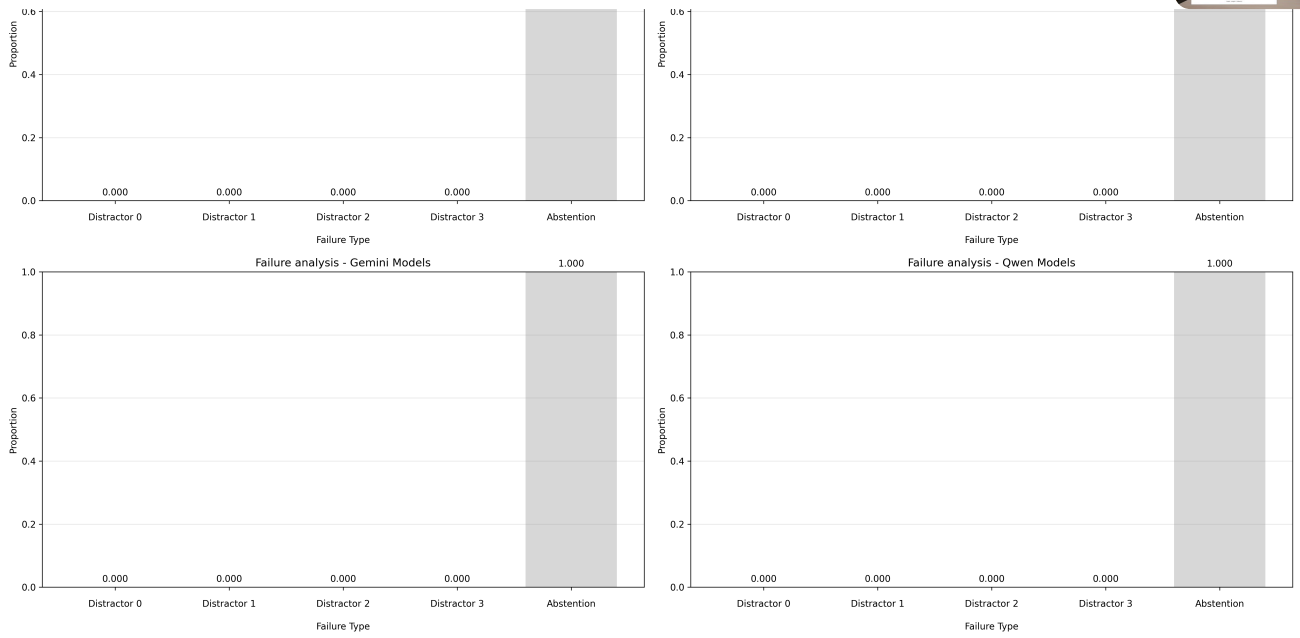


Impact of Distractors: Failure Analysis - arXiv haystack/arXiv needles



Impact of Distractors: Failure Analysis - PG essay haystack/PG essay needles



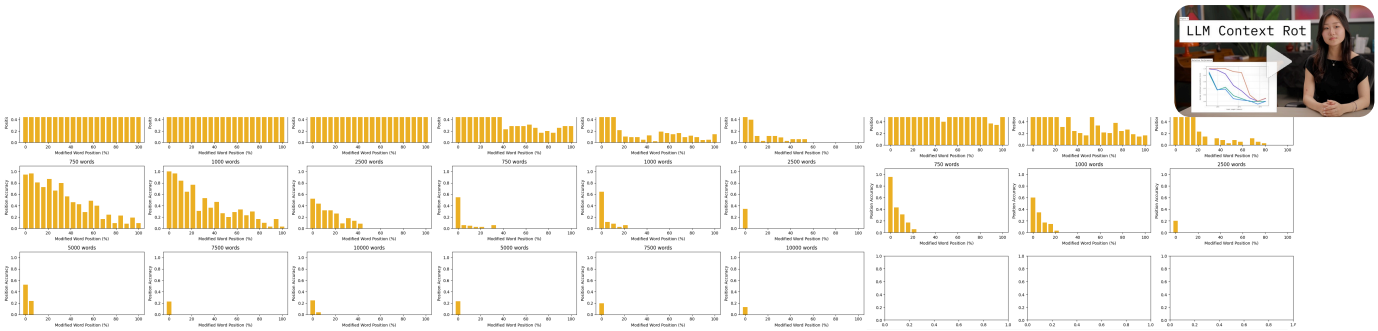


Impact of Distractors: Failure Analysis - PG essay haystack/arXiv needles

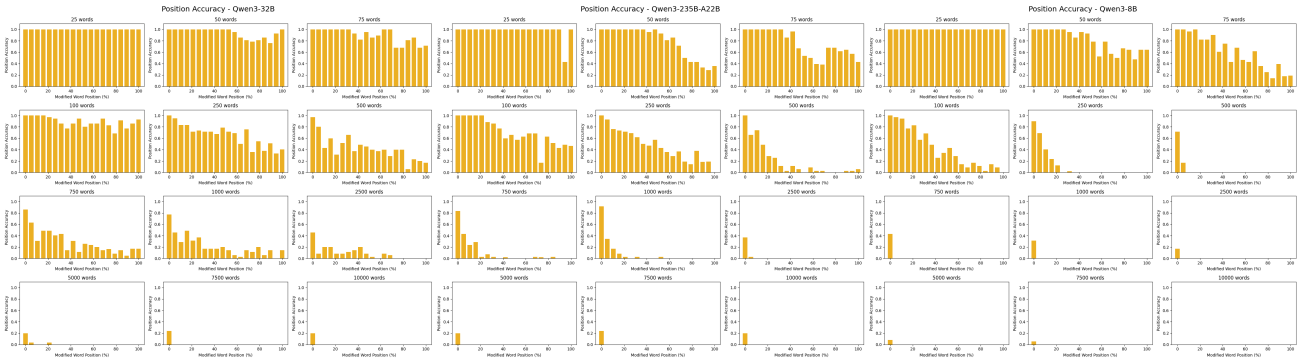
Repeated Words



Repeated Words: Position Accuracy - GPT Family



Repeated Words: Position Accuracy - Gemini Family

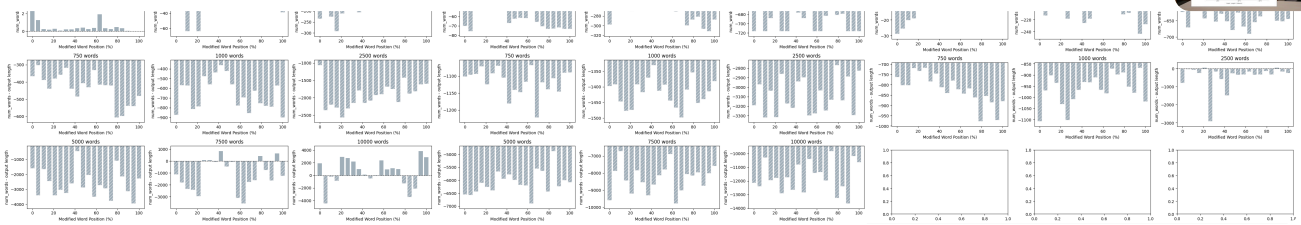


Repeated Words: Position Accuracy - Qwen Family

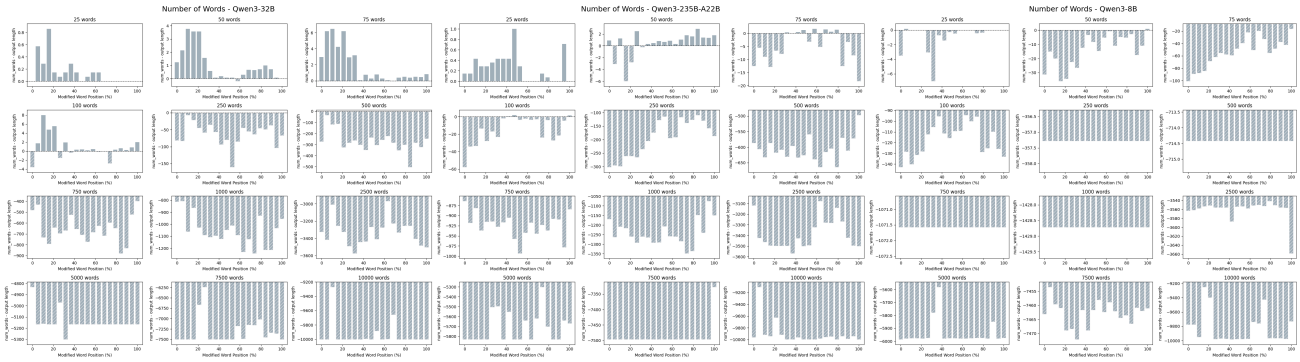


Repeated Words: Word Count Difference - GPT Family





Repeated Words: Word Count Difference - Gemini Family



Repeated Words: Word Count Difference - Qwen Family

Try Chroma Cloud

Chroma is the open-source search and retrieval database for AI applications.

About



Contact Us

Privacy

Terms of Use

2025 Chroma. All rights reserved